
PhoenixInspect Codebase Walkthrough

Architecture, evidence contracts, deterministic IL execution, dump adapters, expression pipelines, hosts, and verification

Repository baseline

github.com/VladimirReshetnikov/PhoenixInspect

Branch: main

Commit: b2148af615d0a0b2694c8515f4beb99151f954f7

Commit date: 6 August 2026 (UTC)

Walkthrough date: 11 August 2026

Prepared for Vladimir Reshetnikov

OpenAI GPT-5.6 Pro

Preface

PhoenixInspect is not easiest to understand by starting at the desktop window, the parser, or even the dump adapter. Its organizing idea lives one level deeper: every answer is a claim about a captured world, and every claim must retain enough identity, evidence, boundedness, and provenance to say exactly what kind of truth it represents. The repository then builds upward from that claim vocabulary: content-derived identities, typed outcomes, a bounded metadata reader, a deterministic IL machine, dump-backed observation adapters, immutable query plans, product facades, and finally two thin user interfaces.

This walkthrough is written for maintainers. It is deliberately code-oriented: project dependencies, important classes, control flow, invariants, failure shapes, test strategy, and concrete recipes for extending the implementation. It also explains several design choices that otherwise look unusually elaborate—for example, why an execution plan is privately issued, why a short memory read is not a Boolean failure, why a source file is withheld when its checksum does not match, and why result status is not one enum.

The analysis is anchored to `b2148af615d0a0b2694c8515f4beb99151f954f7`. Source paths and APIs mentioned in the text refer to that snapshot unless stated otherwise. The repository was inspected through its GitHub source tree, project files, documentation, tests, workflow, and recent commit record. The local document-building environment could not clone GitHub because outbound DNS was unavailable, so this report does not claim an independent local run of the .NET build or test suite. Test counts and hosted verification statements are identified as repository/commit evidence rather than re-executed measurements.

Key idea

A useful one-sentence model is: *PhoenixInspect is a deterministic evidence-composition system that happens to present a debugger-shaped experience.* The debugger resemblance is a user model; the evidence algebra is the architecture.

How to read this walkthrough

For a first architectural pass, read Chapters [1](#), [2](#), [3](#), [7](#), [8](#), and [11](#). For interpreter work, concentrate on Chapters [4](#), [5](#), and [9](#). For expression-language work, read Chapters [8](#) and [6](#) together: parsing and evaluation are only half the problem; authoritative binding is the other half. For user-interface work, Chapter [10](#) explains the strict separation between analysis and presentation. Chapter [13](#) describes the current Edit-and-Continue/Hot Reload frontier at the exact baseline commit.

The appendices are intended for daily use: the project matrix, file atlas, command sheet, outcome vocabulary, and glossary can be consulted without rereading the narrative.

Contents

Preface	i
How to read this walkthrough	ii
1 Orientation: what PhoenixInspect is proving	1
1.1 The product boundary	1
1.1.1 Debugger shape, evidence semantics	1
1.2 The snapshot is the world boundary	2
1.3 The supported product surface	2
1.4 What it is not	3
1.5 The design vocabulary in one diagram	3
1.6 Milestone names embedded in the code	4
2 Repository map, build system, and dependency direction	5
2.1 Top-level geography	5
2.2 The production project graph	6
2.2.1 Project responsibilities	6
2.3 Build configuration	7
2.4 CI as an evidence matrix	8
2.5 Running the code	8
2.6 Status documents versus exact code	9
3 The core contract language	10
3.1 Why the abstractions project is the center of gravity	10
3.2 Result axes: no single status can tell the truth	10
3.2.1 Constructor-enforced coherence	11
3.3 Evidence context	12
3.4 Stable diagnostics	12
3.5 Identity is content first, location second	13

3.5.1	Atomic observations prevent time-slice mixtures	13
3.6	Canonical replay encoding	13
3.6.1	Compatibility discipline	14
3.7	The ECMA-335 type and signature vocabulary	14
3.8	Capability interfaces	14
3.8.1	IValueDomain<TValue>	14
3.8.2	IMemoryModel<TValue, TMemory>	15
3.8.3	Resolution and models	15
3.9	A practical rule for reading contracts	15
4	Whole-graph planning and the deterministic IL machine	16
4.1	Preparation and execution are separate protocols	16
4.2	Graph traversal and accounting	16
4.2.1	Model selection before body acquisition	17
4.3	Machine state versus operational state	17
4.4	The admitted IL profile	17
4.5	Activation	18
4.6	One instruction is one certified transition	18
4.7	Terminal idempotence	18
4.8	Budgets and finite execution	19
4.9	Events and provenance	19
4.10	Failure taxonomy	19
5	Concrete values, persistent memory, and provenance	21
5.1	The concrete domain is a lifted-flat validation domain	21
5.2	Value shapes	21
5.3	Persistent memory	22
5.4	Field loads are evidence-bearing operations	22
5.5	Provenance-enriched concrete values	22
5.5.1	Field and call lineage	22
5.6	Domain laws as executable architecture	23
5.7	Where to extend the domain	23
6	Metadata: from PE bytes to structural authority	24
6.1	Two projects, two responsibilities	24
6.2	IMetadataModule	24
6.3	Opening an SRM module	25

6.4	Deterministic bounds	25
6.5	Projection versus acquisition	25
6.6	Metadata authority in the dump-query product	26
6.7	Complete-table discipline	26
6.8	Signatures and token authority	27
6.9	Portable PDB as a separate authority	27
6.10	Edit-generation contracts at the baseline commit	27
6.11	Common metadata mistakes to avoid	28
7	The ClrMD host adapter and physical evidence	29
7.1	Adapter ownership	29
7.2	Opening a dump	29
7.3	Live attach	30
7.4	Counted raw memory	30
7.5	Module instances	30
7.6	Heap, roots, and object identity	31
7.7	Instance fields and arrays	31
7.8	Frames and variables	31
7.9	Method bodies and execution resolver	32
7.10	Static fields	32
7.11	Portable PDB and source location	32
7.12	Session-level bounds	32
7.13	The dedicated adapter thread	33
7.14	Where adapter bugs usually hide	33
8	Parsing, binding, rooted queries, and constant evaluation	35
8.1	Why Product.DumpQuery is larger than its name	35
8.2	One pinned parser	35
8.3	Versioned expression descriptors	36
8.4	The W2 rooted query	36
8.4.1	Preparation	36
8.4.2	Evaluation	37
8.5	Member chains	37
8.6	Static-field expression pipeline	37
8.6.1	Lazy context acquisition	38
8.6.2	Metadata-to-runtime join	38
8.6.3	Root promotion	38

8.7	The deterministic constant evaluator	39
8.8	Value families	39
8.8.1	Numeric semantics	39
8.8.2	Sequences and LINQ	39
8.8.3	Temporal and BCL values	39
8.8.4	Enums and types	40
8.8.5	Queries and modern C# forms	40
8.9	Canonical identity and display expansion	40
8.10	Expression routing above the project	40
8.11	Failure interpretation	40
9	Counterfactual method evaluation	42
9.1	The product-level handoff	42
9.2	Classification before evidence	42
9.3	Counterfactual request identity	43
9.4	Preparation candidate and private runtime bundle	43
9.5	Private plan issuance	43
9.6	Preparation sequence	43
9.7	Recording memory	44
9.8	Rejecting execution-time resolution	44
9.9	The run loop	44
9.10	Completion and result projection	45
9.11	Why the runner revalidates its own machine	45
9.12	Modeled calls	46
9.13	Counterfactual memory and assumptions	46
10	Inspection services and the three hosts	47
10.1	A deliberately thin presentation layer	47
10.2	Display models	47
10.3	DumpInspectionService	48
10.4	ExpressionEvaluationService	48
10.5	Completion	48
10.6	Source navigation	49
10.7	Process discovery	49
10.8	The console host	49
10.8.1	Entry and exit codes	49
10.8.2	Session commands	50

10.9 The desktop host	50
10.9.1 Watch	51
10.9.2 Immediate	51
10.9.3 Source documents	51
10.10 The headless reference consumer	51
10.11 Presentation-layer anti-patterns	51
11 End-to-end request walkthroughs	53
11.1 Opening a dump	53
11.1.1 User action	53
11.1.2 Control flow	53
11.1.3 Key invariants	53
11.2 Evaluating a fully qualified static field	54
11.3 Using a selected frame for contextual binding	54
11.4 Adopting a root and walking a chain	55
11.5 Composing dump values with constants	55
11.6 Evaluating an admitted method	55
11.7 Opening source from a frame	56
11.8 Reading a non-exact report	56
12 Tests, fixtures, corpus evidence, and CI	58
12.1 Tests are part of the architecture	58
12.2 Unit and model tests	58
12.3 Compiler differential tests	59
12.4 Integration tests over real dumps	59
12.5 Fixture targets	59
12.6 The dump test infrastructure	60
12.7 Meaningful synthetic corpus	60
12.8 Negative evidence and poison cases	60
12.9 Canonical golden tests	61
12.10 Hosted verification at the baseline	61
12.11 What a new feature must prove	61
12.12 Fast debugging by test family	62

13 Edit-and-Continue and Hot Reload: the current frontier	63
13.1 The silent-staleness hazard	63
13.2 Non-negotiable sequence	63
13.3 E1: physical truth gate	64
13.3.1 The base image remains generation zero	64
13.3.2 Even a body edit can extend reference tables	64
13.3.3 ClrMD surfaces can remain pre-edit	64
13.3.4 The original delta blobs are not runtime-retained evidence	64
13.3.5 Filtered dumps remove edit evidence	65
13.3.6 Applied-generation detection is version-pinned	65
13.4 E2: detection and typed non-admission	65
13.5 E3: delta metadata acquisition	66
13.5.1 Generation outcome	66
13.5.2 Lineage chain	66
13.5.3 Measured vocabulary	66
13.6 E4: generation-aware authority composition	66
13.7 E5–E7	67
13.8 Why this work validates the existing architecture	67
14 Extension recipes	68
14.1 Add an IL opcode	68
14.1.1 Steps	68
14.1.2 Tests	69
14.2 Add a pure-call model	69
14.3 Add a constant/BCL operation	70
14.4 Add a new expression syntax form	70
14.5 Add a new dump value kind	71
14.6 Add a ClrMD evidence operation	71
14.7 Add a metadata table family or binding rule	72
14.8 Lift an Edit-and-Continue refusal for one consumer	72
14.9 Add a UI pane	73
14.10 Add a result or canonical field	73
14.11 Update documentation safely	73

15 A maintainer's reading and debugging guide	75
15.1 First-day code-reading itinerary	75
15.2 Search vocabulary	75
15.3 Debugging by stage	76
15.3.1 Wrong or missing static-field result	76
15.3.2 Method preparation failure	76
15.3.3 Unexpected execution invalidity	77
15.3.4 Digest/golden drift	77
15.3.5 Source view mismatch	77
15.3.6 Edited-process problem	77
15.4 Performance reasoning	78
15.5 Code-review checklist	78
15.6 Architectural pressure points	78
15.7 A final mental model	79
A Production project matrix	80
B Key file atlas	83
C Command and workflow reference	85
C.1 Build and preview	85
C.2 CLI startup options	85
C.3 Interactive commands	86
C.4 Capture	86
D Outcome and evidence reference	87
D.1 Core evaluation axes	87
D.2 Common host labels	88
D.3 Adapter evidence	88
D.4 Deterministic bounds	88
E Supported expression surface at the baseline	89
F Glossary	91
G Baseline and source methodology	93
G.1 Exact baseline	93
G.2 Primary material inspected	93
G.3 Repository links	93

Closing perspective	95
List of Figures	96

Orientation: what PhoenixInspect is proving

1.1 The product boundary

PhoenixInspect opens a managed process dump, or attaches to a live .NET process while suspending it, and answers a deliberately bounded subset of C# questions over the captured state. The visible experience is familiar: modules, managed threads, stack frames, locals, heap objects, Watch and Immediate windows, source navigation, and an expression root named `root`. The semantic contract is less familiar and considerably stricter:

- no target thread is resumed to obtain an answer;
- no target method is invoked in the captured process;
- no missing byte is silently replaced with zero, a default value, or an analysis-machine fact;
- no local file path participates in a stable artifact identity;
- no bounded search is presented as exhaustive when its cap was reached;
- no reconstructed source is labelled as the original source;
- no counterfactual execution result is confused with an observation of what historically happened.

These are not merely product copy. They recur as constructor validation, typed unions, explicit semantic modes, content hashes, counted reads, immutable plans, and negative tests throughout the solution.

1.1.1 Debugger shape, evidence semantics

A traditional live debugger has two capabilities PhoenixInspect intentionally lacks: it controls the process, and it can ask the runtime to execute code on its behalf. A dump evaluator instead starts with a fixed and often sparse byte image. The implementation must therefore distinguish at least three kinds of answer:

Observation

A fact read directly from the snapshot or an identity-matching artifact: bytes at an

address, a module row, a runtime field location, a Portable-PDB document record.

Derived query

A deterministic composition of observations: selecting a rooted field, walking a member chain, coalescing a null, binding a static field by metadata name.

Counterfactual execution

The result of interpreting admitted IL against an imported virtual memory state. This answers “what this method computes under the frozen inputs and model policy,” not “what the process executed before capture.”

A fourth mode, abstract analysis, is represented in the core contracts as an architectural direction even where current product paths do not expose a full abstract interpreter.

Invariant

The semantic mode is part of the result, not a comment attached later by a UI. Consequently a host cannot honestly render a counterfactual result as an observation without discarding information the product deliberately supplied.

1.2 The snapshot is the world boundary

For a dump file, the world identity is a SHA-256 digest of the complete dump content. A local path is descriptive provenance only. Reopening byte-identical content elsewhere must produce the same snapshot identity. For a live attach, there is no immutable content to hash; the session instead derives an identity from attach circumstances and keeps the target suspended. Reads replay within that one session, while a fresh attach is intentionally a different world.

The distinction matters everywhere an object, frame, module, or query plan crosses an API boundary. A root selected from snapshot A cannot be evaluated against snapshot B merely because its address and type name happen to match. PhoenixInspect checks snapshot-scoped identities and reports a conflict rather than performing a plausible-looking read from the wrong world.

1.3 The supported product surface

At the baseline commit, the public preview has grown far beyond the original one-field W2 slice. The implemented surface documented and exercised by the repository includes:

- immutable dump files and suspended live-process sessions;
- context-independent fully qualified static fields, plus source-like contextual names when an exact frame and identity-matching Portable PDB are available;
- object-graph member chains of arbitrary admitted depth, conditional access, and null coalescing;
- a deterministic constant evaluator covering the C# numeric tower, strings, chars, arrays, virtual sequences, tuples, anonymous objects, enums, typeof, temporal values, Guid, Version, query expressions, lambdas, patterns, interpolated strings, and selected pure BCL operations;
- composition of exact dump values with folded constants;

- bounded interpretation of a closed IL profile for instance-method expressions, with optional versioned pure-call models;
- heap-root promotion from strong-handle searches or exact static object-reference fields;
- Watch, Immediate, completion, call-stack, locals, modules, heap search, and source panes;
- checksum-verified local source, embedded source, optionally checksum-verified SourceLink content, and an identity-checked ILSpy decompilation fallback explicitly marked as reconstruction;
- complete evidence reports with stable diagnostics, applied bounds, provenance, and replay digests.

This breadth can obscure an important architectural fact: the features do not share one giant evaluator that “tries things.” They are composed from several closed pipelines with different truth modes and result contracts. `ExpressionEvaluationService` and `Dump-ExpressionEvaluator` route among them without erasing those differences.

1.4 What it is not

PhoenixInspect is not a replacement CLR, not a general C# scripting host, and not a promise that arbitrary production dumps are currently supported. The ClrMD adapter is explicit about its walking-skeleton scope: one CLR runtime, named generated/local fixture shapes, same-toolchain CoreCLR layout evidence, bounded catalogs, and selected field/body/value kinds. The expression evaluator may parse modern C# syntax, but admission is project-owned and versioned; a syntactically valid expression can still be an honest `Unsupported` outcome.

Nor does “counterfactual” mean speculative mutation of the dump. The imported memory snapshot is persistent and virtual. Read operations consume captured evidence; any admitted stores create descendant virtual states. Effects are classified separately from evidence and semantic mode.

1.5 The design vocabulary in one diagram

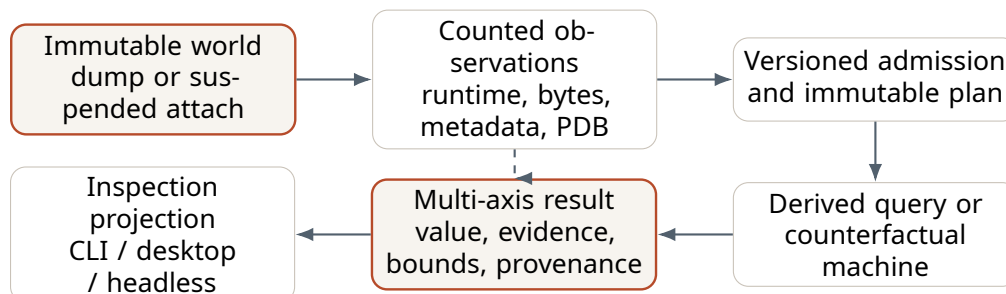


Figure 1.1: The repository’s recurring control-flow shape. Evidence is acquired before it is composed; the final result retains both the composition and its physical inputs.

1.6 Milestone names embedded in the code

Many comments and diagnostic codes use W1–W8 labels. They are historical vertical slices rather than namespaces of a clean-sheet architecture:

Slice	Center of gravity	Lasting architectural contribution
W1	Method-body evidence	Content identities, typed metadata/body acquisition, evidence comparison.
W2	Rooted field query	Parse once, select once, immutable object-specific plan, derived-query result.
W3	Concrete IL execution	Domain-parametric machine, persistent memory, structural handles.
W4	Counterfactual method evaluation	Whole-call-graph preparation, private plan issuance, provenance-bearing execution result.
W5	Expression routing	One product entry path that preserves distinct W2 and W4 contracts.
W6	Member chains	Versioned project-owned chain admission and hop-by-hop evidence composition.
W7	Static-field expressions	Metadata authority, runtime declaration mapping, frame/PDB context, storage observations.
W8	Static-field V2 authority work	Source-anchored metadata catalogs, complete-table discipline, broader binding and conformance machinery.

Later work, notably the E1–E7 Edit-and-Continue plan, builds on those surfaces. A class retaining a W4 or W7 name should therefore be read as an intentionally frozen contract boundary, not as evidence that the overall product stopped at that milestone.

Primary code landmarks

Start with `README.md`, `docs/proposals/architecture/architecture-overview-proposal.md`, and then the public contracts in `src/PhoenixInspect.Core.Abstractions`. The README explains the user-visible surface; the architecture proposal explains the truth model; the contracts show which parts are enforced by code.

Repository map, build system, and dependency direction

2.1 Top-level geography

The repository is arranged around executable evidence rather than around a single application:

src/

Thirteen production projects: core contracts and execution, concrete validation domain, metadata contracts and SRM backend, host contracts and ClrMD adapter, product query/debugging facades, a shared inspection/presentation layer, a headless reference consumer, and CLI/desktop hosts.

tests/

Unit tests, integration tests, dump-generating fixture targets, optimized-build targets, W7/W8 shape fixtures, and the Edit-and-Continue target. Fixtures are part of the evidence apparatus, not incidental sample code.

samples/

The deterministic Contoso.OrderService preview scenario used to produce a narrated, reproducible incident dump.

docs/

Product guides, architecture proposals, milestone history, evidence dispositions, and future-work plans. Some status prose is historical and can lag the exact implementation during an active research sequence; commit-pinned code and the active plan take precedence.

eng/

Headless wrappers, preview-demo automation, documentation checks, and repository governance scripts.

Root build files

PhoenixInspect.sln, global.json, Directory.Build.props, Directory.Packages.props, lock files, and the CI workflow.

2.2 The production project graph

The dependency direction is one of the clearest explanations of the design. Figure 2.1 emphasizes the layering; Appendix A records exact direct references. Lower layers know nothing about dumps or user interfaces. The presentation projects know about immutable display projections, not analysis internals. The product facades sit between physical adapters and hosts.

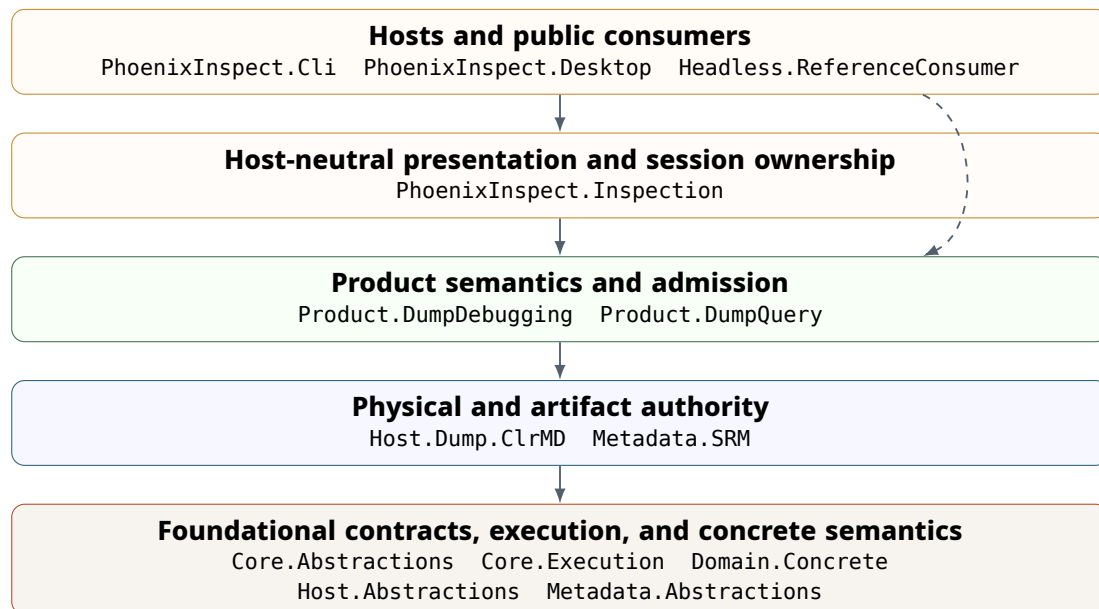


Figure 2.1: Layered production dependency direction. The dashed shortcut represents the headless consumer’s direct use of product APIs; Appendix A lists every exact project reference.

2.2.1 Project responsibilities

Project	Responsibility
PhoenixInspect.Core. Abstractions	Immutable identities, structural type/signature vocabulary, result axes, canonical replay encoding, value-domain and memory-model interfaces, resolution and provenance contracts.
PhoenixInspect.Core. Execution	Whole-method-graph preparation and the deterministic, domain-parametric IL machine.
PhoenixInspect.Domain. Concrete	Lifted-flat concrete values, persistent concrete memory, and provenance-enriched concrete semantics used to validate and productize the execution core.
PhoenixInspect.Host. Abstractions	Snapshot byte-reading contracts that distinguish exact, short, and unavailable reads.
PhoenixInspect.Metadata. Abstractions	Bounded metadata-module capabilities and path-independent module descriptions.

Project	Responsibility
PhoenixInspect.Metadata.SRM	The System.Reflection.Metadata/PE implementation of those metadata contracts, separated into acquisition and projection.
PhoenixInspect.Host.Dump.ClrMD	ClrMD lifetime, module/runtime catalogs, raw memory, heap and field observations, frame/PDB evidence, method-body acquisition, static storage, live attach, and typed adapter outcomes.
PhoenixInspect.Product.DumpQuery	Roslyn parsing and project-owned admission, rooted queries, member chains, static-field binding/evaluation, the deterministic constant evaluator, metadata-authority catalogs, and edit-generation contracts.
PhoenixInspect.Product.DumpDebugging	Unified expression routing for method paths, dump-method acquisition, counterfactual memory/import, preparation, execution, and canonical result projection.
PhoenixInspect.Headless.ReferenceConsumer	Executable public-contract witness, usefulness portfolios, corpus replay, and golden/conformance output independent of a UI.
PhoenixInspect.Inspection	Host-neutral session ownership, immutable display models, evaluation reporting, completion, process discovery, and source navigation. It adds presentation policy, not analysis semantics.
PhoenixInspect.Cli	Scriptable and interactive console host, dump capture command, transcript renderer, command/session state.
PhoenixInspect.Desktop	Avalonia docked debugger-style shell, MVVM state, Watch/Immediate/source panes, and visual interaction.

Key idea

The apparently odd split between `Product.DumpQuery` and `Product.DumpDebugging` is intentional. The former owns read-only binding and derived-query semantics; the latter imports those facts into the IL interpreter and preserves counterfactual semantics. Merging them would make it easier to accidentally present interpreted behavior as observed state.

2.3 Build configuration

The repository pins the .NET SDK to 10.0.201 with latest-patch roll-forward and disallows prerelease SDK selection. Shared build properties establish:

- target framework `net10.0`;
- C# 14;
- nullable reference types and implicit global usings;
- deterministic builds and XML documentation;
- package lock files everywhere and locked restore in CI;

- warnings as errors in CI rather than necessarily during every local exploratory build.

A second `src/Directory.Build.props` scopes product versioning (`0.1.0-preview.1`) to shipping projects. Keeping fixture assemblies outside that version injection is significant: fixture identity is evidence, so incidental product-version changes must not silently perturb the binaries and dumps used as controls.

Central package management fixes the major moving parts: ClrMD 4.0, the diagnostics client, Roslyn 5.3.0, Avalonia 12.1, Dock, AvaloniaEdit, ProDataGrid, ILSpy/decompiler 9.1, and the xUnit/test infrastructure. The parser also records the Roslyn package version in its own profile identity; the dependency version is therefore part of semantics, not merely a build detail.

2.4 CI as an evidence matrix

The workflow is split by what each lane proves:

Documentation lane

Runs link validation and a guard that checks the documented headless workflow. This prevents the architectural narrative from becoming completely detached from executable entry points.

Fast Windows lane

Locked restore, Release build, unit tests, and the fast integration category.

Real dump evidence lane

Generates/opens ordinary fixture dumps and exercises adapter/product paths against actual captured state.

Preview demo lane

Executes the end-to-end sample workflow and publishes the narrated transcript as an artifact.

Optimized-context lane

Uses a Release/optimized target to prove behavior where debugger context is incomplete or transformed by optimization.

The latest commit record for `b2148af6` reports 1,327 unit tests, 629 fast integration tests, 12 Edit-and-Continue fixture tests, four focused delta-acquisition tests, and zero skips for the cited closure runs. Those are commit-recorded verification facts, not independent measurements made while producing this document.

2.5 Running the code

The shortest user-facing path is the preview script:

```
./eng/Invoke-PreviewDemo.ps1
```

To inspect an existing dump through the CLI:

```
dotnet run --project src/PhoenixInspect.Cli -- <path-to-your.dmp>
```

The CLI also accepts a suspended live attach, repeated `--eval` or `--command` arguments, a script file, verbose evidence, and color suppression. The desktop project uses the same `PhoenixInspect.Inspection` services, so the choice of host does not select a different evaluator.

2.6 Status documents versus exact code

A research repository can legitimately contain several time scales. At this baseline, portions of the top-level README still describe W8.2 as active, while the exact tree and recent plan history already contain completed E1–E3 Edit-and-Continue work and name E4 as the next stride. The correct reading order for a current-state question is:

1. exact commit-pinned code and tests;
2. the active plan’s status paragraphs and closure commit;
3. milestone history/evidence dispositions;
4. general README status prose.

The older text remains useful as architectural history. It should not be silently “corrected” when documenting a past milestone, but a current walkthrough must call out the skew.

Primary code landmarks

Build and dependency landmarks: `global.json`, `Directory.Build.props`, `src/Directory.Build.props`, `Directory.Packages.props`, `PhoenixInspect.sln`, and `.github/workflows/ci.yml`.

The core contract language

3.1 Why the abstractions project is the center of gravity

`PhoenixInspect.Core.Abstractions` contains no dump reader and no execution loop, yet it is the most important project for understanding repository-wide behavior. It defines what can be claimed. The rest of the system either produces these claims, transforms them, or displays them.

Its contents fall into five families:

1. structural identities and handles for artifacts, modules, methods, fields, objects, and source worlds;
2. a bounded ECMA-335 type/signature vocabulary;
3. multi-axis evaluation results, diagnostics, provenance, evidence context, and deterministic bounds;
4. replay/canonical-encoding primitives;
5. capability interfaces for value domains, memory, budgets, metadata resolution, and pure-call models.

This arrangement prevents lower-level mechanics from inventing ad hoc meanings. A short memory read, ambiguous metadata match, unsupported signature, budget stop, and invalid program are different facts before any host sees them.

3.2 Result axes: no single status can tell the truth

The generic `EvaluationResult<T>` is best understood as a product type rather than a success/failure union. Its principal axes are independent, subject to coherence constraints.

Axis	Representative values	Question answered
Semantic mode	Observation; DerivedQuery; CounterfactualExecution; AbstractAnalysis	What kind of truth is being claimed?
Completion	Completed; Blocked; BudgetExhausted; Cancelled; Invalid	Why did the operation stop?
Completeness	Complete; Partial; None	How much semantic answer exists?
Evidence	Exact; Partial; Unavailable; Conflict; Invalid	How strong and coherent were the physical inputs?
Effects	None; VirtualOnly; Modeled; Unsupported	What effect semantics were admitted?
Value	Optional typed value	Is there a semantic value, prefix, or witness to carry?
Provenance	DumpMemory; RuntimeStructure; Artifact; Policy; Transformation	Which sources and transformations support the answer?
Diagnostics	Stable code plus artifact-independent message	What named disposition explains a non-ideal outcome?
Reached bounds	Named deterministic limits actually encountered	Which finite policy gates materially constrained the operation?

Consider three superficially similar “no ordinary value” outcomes:

- a syntactically valid but unsupported expression: completion may be Blocked, completeness None, evidence Exact; the parser knows exactly that the admitted profile excludes it;
- a field read whose dump bytes are truncated: completion Blocked, completeness None or Partial, evidence Partial;
- a cancelled method interpretation after several transfers: completion Cancelled, completeness Partial, value an execution-prefix witness, and exact/partial evidence aggregated from the observations already consumed.

Flattening all three to “error” would lose the information PhoenixInspect exists to preserve.

3.2.1 Constructor-enforced coherence

The result constructor validates cross-axis invariants. In schematic form:

Listing 3.1: Representative coherence rules enforced by the result contract. This is schematic rather than a verbatim source extract.

```
1 if (mode is Observation or DerivedQuery)
2     require effects == None;
3
4 if (completeness is Complete or Partial)
5     require value is present;
6
7 if (completeness == None)
8     require value is absent;
9
10 if (completion != Completed)
11     forbid completeness == Complete;
```

The important architectural consequence is that a malformed result cannot travel far. A product facade cannot claim a complete value after cancellation, and a derived query cannot accidentally announce virtual side effects, without failing at construction.

3.3 Evidence context

EvaluationEvidenceContext records the identity and finite policy envelope surrounding a result. Its source kind distinguishes no external source, a dump snapshot, an artifact, or synthetic evidence. Identity availability is itself explicit: Available, Unavailable, or NotApplicable. This avoids the common ambiguity where a null digest could mean either “not needed” or “needed but missing.”

Fallback is similarly explicit. The context says either that no fallback occurred or names the fallback that did. If a disk assembly was used only after its content identity matched the dump module, that transformation belongs in provenance; it is not an invisible implementation convenience.

Only bounds actually reached are retained. A configured maximum that played no role in an operation is not evidence about the outcome. This keeps replay identity stable and reports readable while still making cap-induced incompleteness auditable.

Invariant

A result over a dump snapshot must name the snapshot identity. Where module identity is relevant, the context must state either the exact module identity or an explicit unavailable outcome. “We happened not to populate the field” is not an admissible third state.

3.4 Stable diagnostics

Diagnostics consist of a stable code and an artifact-independent explanation. Paths, exception payloads, arbitrary parser text, and target-specific display strings are excluded from canonical identity. Hosts may add contextual presentation around the diagnostic, but the product code remains suitable for replay and golden comparison.

The code families also reveal pipeline ownership: META_* for metadata admission, DUMP_* for host evidence, QUERY_* for rooted query syntax/evaluation, W4.* for counterfactual preparation/execution, and newer W7/W8 or Edit-and-Continue dispositions for those staged contracts. The names are intentionally more precise than an exception type.

3.5 Identity is content first, location second

Several identity types coexist because they answer different questions:

ArtifactContentIdentity

Is this complete external file byte-for-byte the same artifact?

ModuleContentIdentity

Does this metadata content, anchored by MVID and metadata bytes, identify the same logical module content?

ModuleId

What module identity results from content identity, PE stamp, and artifact identity?

ModuleHandle

What compact structural identity should the execution core use without knowing paths or SRM objects?

MethodHandle/FieldHandle

Which metadata definition in which structural module is referenced?

Snapshot identity

Which captured world supplied this address, runtime object, or frame?

Paths remain in descriptors and display provenance. They can help locate a candidate artifact but never prove that candidate. This is why copying an assembly or dump does not change its stable identity, while replacing a file at the same path does.

3.5.1 Atomic observations prevent time-slice mixtures

`IMetadataModule.GetMethodDefinition` returns body and activation shape as one immutable observation. The alternative—fetching a body, then separately fetching signature and locals—would permit callers to combine facts acquired from different effective generations or sources. The interface still exposes a body-only compatibility operation for W1 evidence comparison, but interpreter activation is required to use the atomic definition.

This distinction becomes especially important in the Edit-and-Continue work, where a base-image body and a generation-aware signature would be individually plausible yet jointly false.

3.6 Canonical replay encoding

`CanonicalReplayEncoding` is a small serialization protocol whose job is not interchange convenience but deterministic identity. A writer begins with a domain string and positive schema version, then emits:

- integers in an explicitly chosen big-endian representation;
- length-prefixed byte sequences;
- strings as counted UTF-16 code units rather than environment-dependent text encodings;
- normalized digest text and bounded vectors in deterministic order.

The resulting bytes are hashed with SHA-256. Equality for many immutable contracts compares canonical bytes, and deterministic hash codes derive from the digest rather than object identity.

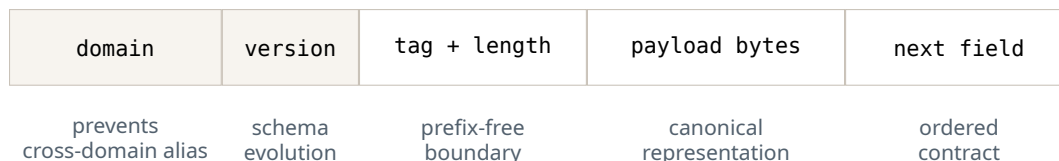


Figure 3.1: Conceptual shape of canonical contract encoding. The exact writer API is typed, but every sequence is domain-separated, versioned, and prefix-free.

3.6.1 Compatibility discipline

Canonical encodings are treated as compatibility surfaces. Additive fields are normally appended behind a distinct tag or absence-preserving representation so an old zero-feature case remains byte-identical. The Edit-and-Continue work repeatedly tests this requirement: a zero-generation module must keep existing W1–W8 digests unchanged. A change that merely preserves object equality but changes canonical bytes is therefore not harmless. It can invalidate replay identity, goldens, corpus reports, and cross-run comparison.

3.7 The ECMA-335 type and signature vocabulary

The core contracts include a bounded structural grammar for CLI types and signatures rather than leaking `System.Type`, SRM handles, or reflection objects into execution. Representative shapes include primitive stack kinds, named value/reference types, arrays, pointers/byrefs where admitted, generic parameters and instantiations, and method/local/field signatures.

The large `BoundedEcmaSignatureProjection.cs` implementation performs counted, closed-profile decoding. It is intentionally not a permissive ECMA metadata parser. Unsupported constructs remain typed unsupported results; malformed encodings remain invalid; recursion, node count, and token resolution are bounded.

This has three benefits:

1. metadata backends can change while execution identities stay structural;
2. tests can generate and compare signatures without loading assemblies;
3. the machine can validate stack and argument shapes without depending on reflection or the host runtime.

3.8 Capability interfaces

3.8.1 `IValueDomain<TValue>`

The value domain supplies lattice and execution semantics: bottom/top, constants, join/meet/widen, partial order, static type and stack kind, exact-value extraction, and

the admitted arithmetic operations. The machine does not know whether a value is concrete, abstract, provenance-wrapped, or symbolic.

This is more than generic-programming elegance. It separates the IL transfer relation from the precision policy. The same instruction can produce an exact integer in the concrete domain or an explained unknown in another domain without changing the machine's control-flow code.

3.8.2 `IMemoryModel<TValue, TMemory>`

Memory is persistent: reads return a typed `MemoryLoadResult`; writes return descendant snapshots. The contract explicitly forbids turning missing dump evidence into default field values. A zero-initialized virtual allocation and an unavailable captured field are semantically different and must remain distinguishable.

3.8.3 Resolution and models

Resolution services translate structural handles and tokens into admitted method/field definitions. Pure-call model registries provide explicit, versioned replacements for selected helpers. Model selection occurs during graph preparation, before execution and, where required, before acquiring a target body. That order ensures the frozen graph says exactly which capability will answer each call site.

3.9 A practical rule for reading contracts

When a class seems to carry “too many” fields, classify them into four groups:

1. semantic payload;
2. identity/replay payload;
3. physical evidence and provenance;
4. finite-policy accounting.

Most apparent redundancy disappears. For example, a result may retain both a selected field descriptor and raw read evidence because the former says *what was selected* while the latter says *what bytes made the value available*.

Primary code landmarks

Read these files together: `EvaluationResult.cs`, `EvaluationEvidenceContext.cs`, `CanonicalReplayEncoding.cs`, `IValueDomain.cs`, `IMemoryModel.cs`, and `BoundedEcmaSignatureProjection.cs`. The unit suites `EvaluationResultTests.cs`, `CanonicalReplayEncodingTests.cs`, and `BoundedEcmaSignatureProjectionTests.cs` are the quickest executable specification of edge cases.

Whole-graph planning and the deterministic IL machine

4.1 Preparation and execution are separate protocols

The execution core does not accept an arbitrary method handle and resolve as it goes. `MethodGraphPlanner` first discovers, validates, and freezes the complete admitted direct-call closure. `ILMachine<TValue, TMemory>` then executes only that frozen graph. This separation is the execution analogue of parse-once/select-once query planning.

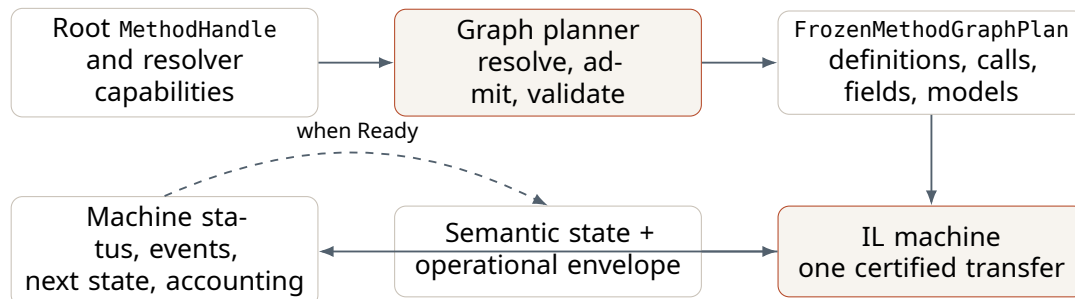


Figure 4.1: The plan/execute split. No metadata or call-target resolution is permitted in the execution loop.

The planner's output is not simply a cache. It is an admission certificate: every reachable body, signature, local, field operand, call target, model selection, and required logical depth has already passed a closed set of checks.

4.2 Graph traversal and accounting

Planning performs root-first depth-first traversal, visiting call sites in increasing IL offset. Public vectors are later canonicalized by structural identity so replay does not depend on incidental dictionary order. A fresh first-result cache is scoped to one preparation attempt, which prevents a previous request's failure or capability observation from contaminating another request.

The traversal has explicit limits, including a product-level traversal-unit budget and an internal method-count cap. Importantly, a zero traversal limit stops before invoking resolver or model-registry capabilities. Tests verify not only the final disposition but which capability calls were avoided.

Each potential operation is charged according to a frozen accounting rule. When a bound is crossed, the failure retains the first rejected charge and the traversal prefix rather than pretending no work occurred.

4.2.1 Model selection before body acquisition

For an optional required pure model, the planner identifies the exact target and selects the model before asking for that callee's body. This ordering serves two purposes:

- the model is a declared semantic replacement, not a fallback after body acquisition happened to fail;
- a modeled external/helper call can be admitted without granting unnecessary body-reading authority.

An interpreted-only request never queries the optional model registry. Capability non-use is part of the contract and is tested.

4.3 Machine state versus operational state

The machine maintains two parallel envelopes:

Semantic state

Call stack, frame arguments/locals/evaluation stack, persistent memory, root return value, and terminal target-exception state.

Operational state

Remaining instruction budget, configured and observed logical depth, active-frame high-water marks, pure-model attempts, invocation counts, and completed modeled-call counts.

Separating them prevents budget instrumentation from changing semantic equality and makes “no instruction executed” assertions precise. A budget-exhausted transfer returns the exact same semantic state and operational object when the pre-instruction budget is zero.

4.4 The admitted IL profile

The current machine is intentionally a closed, scenario-derived subset rather than a partial implementation that silently mishandles the rest of ECMA-335. Its concrete path includes the operations needed by the proven fixture corpus: integer constants and arguments, initialized locals, selected arithmetic, ordinary instance field loads, control-neutral `nop`, `return`, direct admitted calls, and pure modeled calls. Additional functionality is split between transfer files and the prepared-graph implementation, but every opcode must have explicit validation, stack behavior, event behavior, accounting, and tests.

Unsupported opcode or signature shapes fail during planning where possible. Invalid stack behavior and malformed bodies remain invalid-program outcomes. Unknown continuation is admitted only under an explicit `UnknownExecutionPolicy` and only when the value domain exposes the needed capability.

Boundary or caveat

Do not infer full CLR semantics from the generic class name `ILMachine`. It is a deterministic machine for a closed profile. The design makes extending that profile disciplined; it does not claim that an unimplemented opcode can be approximated safely.

4.5 Activation

Prepared activation verifies the root method, required logical depth, receiver/argument vector, static types, locals, and initial memory identity. It creates one root frame at IL offset zero with an empty evaluation stack and default-initialized locals according to the admitted semantics. Activation consumes no instruction budget.

The counterfactual runner later revalidates this state: exactly one root frame, no return value, no terminal exception, same persistent memory identity, matching root method, and exact reference retention for the materialized argument vector.

4.6 One instruction is one certified transition

`StepOne` returns a `StepOutcome` stamped by the machine for the exact prior semantic and operational envelopes. The product runner verifies that stamp before trusting any result. Successful ordinary transfers:

- consume exactly one instruction unit;
- produce an `InstructionExecuted` event, optionally followed by one frame or precision event;
- preserve persistent memory identity for the current read-only rooted product path;
- change call-stack depth only as announced by a frame event;
- preserve or extend model-attempt accounting by at most one coherent entry.

Blocked and invalid transfers change neither semantic state nor budget and emit no ordinary events. Target exceptions have their own terminal union and event shape. The runner treats a violation of these internal contracts as a replay-invalid result rather than accepting a plausible next state.

4.7 Terminal idempotence

After a completed root return, the runner calls `StepOne` once more and requires an idempotent terminal transition. It also checks that no field load, provenance node, budget, event, or model counter changed. This is a compact but powerful replay guarantee:

terminal state is not merely recognized by one control branch; it is stable under the machine's public stepping protocol.

4.8 Budgets and finite execution

The machine's instruction budget is caller-configured within product limits. The runner also calculates a graph-derived finite dynamic instruction bound for the acyclic prepared graph and caps the number of step requests at the lesser of that cost and the instruction budget, plus the mandatory terminal check. Exceeding that bound is treated as internal invalidity: the frozen graph and transfer accounting promised finiteness.

Logical call depth is separate from active frame depth. A pure model may enter logical depth without creating an interpreted frame, so both high-water marks are retained. Preparation rejects a configured logical-depth limit smaller than the graph's required depth before any instruction executes.

Cancellation is observed only at ready machine boundaries. A terminal transfer and its required idempotence check take precedence over cancellation arriving after completion. A cancelled run with no executed instruction carries no semantic value; a cancelled run after progress can carry an execution-prefix value and partial completeness.

4.9 Events and provenance

Debug events are not a UI log pasted onto execution. They are part of the transition contract and canonical result. Representative events describe instructions, frame pushes/pops, value precision loss, and target exceptions. Field-load evidence and provenance lineage connect values back to captured observations and transformations.

Exact primitive values can bypass expensive lineage structures where no explanation is needed. Explained unknowns require provenance capability so the product can report why precision was lost instead of returning an opaque top value.

4.10 Failure taxonomy

A useful debugging distinction is where a failure occurred:

Resolution/admission failure

The graph could not be completely frozen. No partial executable plan escapes.

Request/binding failure

Receiver, arguments, evidence, policy, or memory/domain capabilities are mutually incompatible.

Activation failure

A supposedly valid plan could not produce the exact initial machine state.

Blocked transfer

A required admitted runtime capability was unavailable at one instruction boundary.

Invalid program/transition

IL or a capability result contradicted the closed machine contract.

Budget/cancellation

Finite host policy ended an otherwise coherent run.

Target exception

The modeled target semantics produced an exception; the current non-null rooted facade has deliberately narrow handling for this case.

Because each stage has typed output, a test can assert that a malformed call signature never reaches activation, or that zero traversal never touches a resolver, rather than merely observing a final error code.

Primary code landmarks

Core execution landmarks: `MethodGraphPlanner.cs`, `MethodPlanBuilder.cs`, `FrozenMethodGraphPlan.cs`, `ILMachine.cs`, `ILMachine.PreparedGraph.cs`, and `ILMachine.Transfers.cs`. The deepest behavioral specifications are `MethodGraphPlannerTests.cs`, `MethodGraphPlannerTraversalAccountingTests.cs`, `ILMachineTests.cs`, `PreparedGraphExecutionTests.cs`, and `PreparedGraphModelExecutionTests.cs`.

Concrete values, persistent memory, and provenance

5.1 The concrete domain is a lifted-flat validation domain

`PhoenixInspect.Domain.Concrete` supplies the domain used by current counterfactual product paths. Its lattice is intentionally simple and mechanically testable. For a fixed static type, the ordering is essentially

$$\perp \sqsubseteq c \sqsubseteq \top,$$

where each exact constant c is incomparable with a distinct exact constant. Consequently:

- joining an exact value with itself preserves it;
- joining distinct exact values yields unknown/top;
- meeting distinct exact values yields bottom;
- bottom joins to the other operand and top meets to the other operand;
- widening is predictable because the domain has no ascending numeric intervals.

This is not intended to be the final abstract-analysis domain. It is the smallest domain that can validate transfer semantics, preserve exact fixture values, and represent explained loss of precision.

5.2 Value shapes

`ConcreteValue` represents exact primitive values and deterministic structural references admitted by the machine. Integer operations use the intended unchecked `Int32/Int64` semantics where the opcode profile calls for them. Object and array references are identities in virtual/captured memory, not analysis-process object references.

Static type and CLI stack kind are kept explicit. A value that is structurally `Int32` cannot be substituted for a reference simply because both are stored in one C# union type. Argument and field-load validation use those shapes before execution.

5.3 Persistent memory

`ConcreteMemory` is an immutable snapshot. `ConcreteMemoryModel` reads and returns either exact values or typed non-exact loads; writes construct descendant snapshots. Persistence has several consequences:

- a plan can retain its exact initial memory without worrying that another run mutates it;
- replay comparison can use structural memory identity;
- virtual effects are distinguishable from captured evidence;
- machine states can share unchanged memory cheaply;
- a failed transfer can prove it preserved the exact prior memory object.

The dump product currently emphasizes read-only rooted evaluation. The persistent interface nevertheless keeps future virtual mutation and stepping semantics honest: a store never writes into the dump, and descendants are explicit.

5.4 Field loads are evidence-bearing operations

A field load is not simply `TValue?`. It can be exact, absent under a proved condition, unavailable, partial, conflicting, or invalid, with evidence and diagnostics. The memory model must not use CLI default initialization to fill a hole in captured memory. Default initialization is valid for a newly allocated virtual object whose construction semantics are known; it is not valid for a dump page that was omitted.

The counterfactual product wraps the memory model in `CounterfactualRecordingMemoryModel`. This adapter records which predeclared field observations were actually reached and in what ordinal order. Final evidence is aggregated from reached observations, not from every field the graph could theoretically access.

5.5 Provenance-enriched concrete values

`ProvenanceConcreteValue` pairs concrete precision with a lineage graph. Nodes can represent dump memory, field transfers, calls, model transformations, and precision loss. The graph is interned to keep repeated lineage structures deterministic and bounded.

The design avoids attaching a heavyweight explanation to every exact constant. Where an operation remains exact and its source is already represented by the result-level evidence, the value can remain compact. When values merge or a capability returns an explained unknown, the lineage records the cause.

5.5.1 Field and call lineage

Separate test families cover field-source lineage, field-transfer lineage, machine propagation, interpreted-call lineage, and modeled-call lineage. This granularity is useful when extending the machine: an arithmetic opcode may compute the correct number while accidentally dropping one operand's provenance. A conventional value-only test would miss that regression.

5.6 Domain laws as executable architecture

`ConcreteDomainLawTests.cs` checks lattice identities and order relations.

`ConcreteMemoryModelTests.cs` checks persistence and read/write behavior. Provenance tests check canonical lineage rather than just display strings. These suites matter because the interpreter assumes algebraic laws: if join is not idempotent or memory descendants mutate ancestors, machine-level replay guarantees collapse.

Key idea

The concrete domain is intentionally boring mathematics. That is a virtue: it gives the product an exact baseline against which more ambitious abstract or symbolic domains can later be validated.

5.7 Where to extend the domain

A new value category normally requires coordinated changes in four places:

1. structural type/stack-kind representation in core contracts;
2. concrete value construction, equality, and canonical representation;
3. domain transfer semantics and exact-value extraction;
4. memory/model and provenance tests demonstrating persistence and lineage.

Do not add an opaque “object” payload holding analysis-machine instances. That would make identity, canonical replay, and target-platform semantics ambient.

Primary code landmarks

Read `ConcreteValue.cs`, `ConcreteDomain.cs`, `ConcreteMemory.cs`, `ConcreteMemoryModel.cs`, and the provenance wrapper/lineage files. Then read `ConcreteDomainLawTests.cs`, `ConcreteMemoryModelTests.cs`, and the `Provenance*Tests.cs` families.

Metadata: from PE bytes to structural authority

6.1 Two projects, two responsibilities

Metadata work is split between two projects. `PhoenixInspect.Metadata.Abstractions` defines the bounded capability the rest of the system may rely on. `PhoenixInspect.Metadata.SRM` implements that capability over managed PE and metadata bytes using `System.Reflection.Metadata` and `PEReader`.

The split is essential because metadata bytes can come from more than one place:

- a local PE artifact opened from disk;
- counted metadata bytes read from dump memory;
- a caller-supplied Edit-and-Continue metadata delta;
- eventually, a generation-aware composition of base and delta tables.

Acquisition answers “where did the bytes come from and are they complete?” Projection answers “what structural method, field, type, or signature do those bytes declare?” Combining the two would force dump-backed code either to duplicate SRM semantics or to masquerade memory bytes as a file.

6.2 `IMetadataModule`

The central abstraction exposes:

- a canonical path-independent `ModuleId`;
- a non-identity `ModuleDescriptor` containing display name and artifact location evidence;
- a structural `ModuleHandle` for the execution core;
- `MethodDef` token validation and conversion to `MethodHandle`;
- atomic method-definition projection, including body and activation shape;
- body-independent method-operand resolution;

- contextual same-module FieldDef resolution;
- a body-only compatibility read for evidence comparison.

The contract intentionally excludes name-based overload guessing, ambient assembly loading, and broad MemberRef/MethodSpec resolution from the current admitted profile. A production binder must include namespace, arity, and signature and must report conflicts rather than choose the first method with a matching simple name.

6.3 Opening an SRM module

`SrmMetadataModule` opens a managed PE through a bounded, typed path. The throwing constructor exists for named fixtures and programmer-controlled inputs. The static `Open` method maps external failures to stable resolution outcomes:

- malformed or structurally invalid path — invalid;
- missing/unreadable artifact — unavailable;
- file larger than the deterministic external-input cap — unsupported;
- malformed PE/metadata — invalid.

It does not echo arbitrary parser or file-system exception payloads into the product result.

The module identity combines:

1. complete artifact SHA-256 identity;
2. metadata content identity derived from MVID and metadata bytes;
3. PE timestamp and image size as a structural stamp.

The path and filename remain in the descriptor. Byte-identical copies therefore produce the same module and execution handles.

6.4 Deterministic bounds

The backend declares hard caps, including a 512 MiB external artifact limit, 100,000 type definitions for bounded fixture lookup, 1,000,000 method definitions, bounded lookup-name length, and a 4 KiB method-body code-byte profile for the current execution slice. These limits are not hidden implementation details. Crossing one produces a named unsupported or partial disposition and, where applicable, a reached-bound record.

The simple type/method lookup helper is documented as fixture-oriented. It counts all matches and reports ambiguity. This is a recurring repository pattern: convenience APIs are explicitly scoped so they do not become accidental production binding rules.

6.5 Projection versus acquisition

`SrmMetadataProjection` contains the reusable transformation from SRM rows and signatures to PhoenixInspect structural contracts. Both disk-backed modules and dump-counted metadata can invoke the same projection logic. This is crucial for differential

tests: if the dump path and disk path disagree, the discrepancy can be localized to byte acquisition or runtime mapping rather than duplicated metadata semantics.

Projection validates token kind/range, method implementation kind, signatures, local layouts, field ownership, and the closed same-module call/field profile. A successful `ResolvedMethodDefinition` is therefore an activation-ready observation, not a thin wrapper around an SRM handle.

6.6 Metadata authority in the dump-query product

Static-field binding requires more than the minimal method-oriented `IMetadataModule`. `PhoenixInspect.Product.DumpQuery` contains a larger source-anchored authority system: complete-table catalogs, declaration identities, ownership chains, type ancestry, signatures, constraints, imports, aliases, and binding outcomes. The W8 work emphasizes complete physical tables and canonical source correlation rather than “find a row that looks right.”

The authority pipeline generally follows this order:

1. acquire complete counted metadata bytes for a runtime module instance;
2. produce physical table catalogs retaining row order, row counts, and source identity;
3. project definition identities and ownership/ancestry relationships;
4. bind a parsed spelling under a closed contextual or fully qualified route;
5. join the selected metadata declaration back to exactly one runtime module instance;
6. map the metadata field to runtime storage geometry;
7. read and semantically decode storage.

The separation between metadata declaration and runtime declaration is intentional. A `FieldDef` token proves what the module declares; it does not by itself prove which runtime static base or thread/context-specific storage contains the captured value.

6.7 Complete-table discipline

Several authority contracts retain entire table families instead of issuing a partial list of interesting rows. This provides a proof boundary for absence and ambiguity:

- a complete catalog can prove a name is absent within the admitted module set;
- row identifiers remain meaningful because every physical RID is represented or a typed stop explains why not;
- superseded/edit-added rows can later be composed without losing base history;
- bounds use cap-plus-one accounting so “exactly at cap” and “there was at least one more” remain distinct.

A partial scan may still be useful, but it cannot claim exhaustive absence. That distinction flows into binding and eventually the UI’s Absent versus Partial/Stopped rendering.

6.8 Signatures and token authority

Field and method signatures are decoded through the bounded core grammar. Token-bearing signature nodes are resolved against the exact catalog and module source that supplied the signature. A matching display name in another assembly is not a substitute. Type forwarding, assembly references, generic instantiation, variance, and assignability are admitted only where the current authority contracts have exact support.

For enum constants and literal fields, the constant evaluator reads the declaring module's metadata Constant table. It does not use the analysis runtime's reflection view of a same-named enum. This is a subtle but essential guarantee: an old dump can be inspected on a machine with newer framework assemblies without silently changing the enum member map.

6.9 Portable PDB as a separate authority

Portable PDB data is never assumed to be part of module metadata. The dump adapter first validates a candidate artifact against the module's CodeView identity. The expression-context pipeline then projects namespaces, imports, aliases, and lexical envelopes under their own bounds and completeness certificates.

A source path inside the PDB is a recorded string, not proof that the current local file is the captured source. The inspection layer performs content verification later, as described in Chapter 10.

6.10 Edit-generation contracts at the baseline commit

The latest metadata frontier is `MetadataEditGenerationContracts.cs`. It formalizes what can be claimed about caller-supplied Edit-and-Continue metadata deltas after physical E1 measurements showed that the runtime does not retain the original delta blob in ordinary dump memory.

`MetadataEditGenerationOutcome.Acquire`:

1. requires a positive caller-declared generation ordinal and initialized bytes;
2. computes the blob's SHA-256 immediately;
3. enforces a 1 MiB maximum before decoding;
4. opens the bytes as a metadata image;
5. reads the Module row's generation number, MVID, edit ID, and base edit ID;
6. retains every `EncLog` token/operation row and every `EncMap` token in physical order;
7. returns one exact outcome or one prefix-free typed stop such as unreadable blob, generation mismatch, empty edit ID, or byte-count bound.

The lineage-chain contract then joins generations by measured rules:

- every generation shares the baseline MVID;
- generation one has an empty base edit ID because generation zero has no edit ID;
- each later generation's base edit ID equals its predecessor's edit ID;
- an optional physically acquired module edit state must agree with the supplied chain length;

- disagreement is a conflict, not a preference for either source.



Figure 6.1: Measured Edit-and-Continue lineage relation retained by the E3 contracts.

E3 stops at physical delta acquisition and lineage. It does not yet project effective logical metadata tables; that is E4's job. This sequencing avoids a dangerous shortcut where a consumer starts using delta rows without a complete and replayable composition model.

6.11 Common metadata mistakes to avoid

- Treating MVID alone as complete artifact identity.
- Letting a path or simple assembly name decide a join.
- Re-resolving a method token during execution after preparation froze a different definition.
- Returning the first same-name member instead of counting conflicts.
- Calling a partial table scan “not found.”
- Using analysis-runtime reflection for target enum/type facts.
- Combining a generation-zero body with generation-aware metadata.
- Dropping physical rows after composing an effective view; history is part of replay identity.

Primary code landmarks

Primary files: `Metadata.Abstractions/IMetadataModule.cs`, `Metadata.SRM/SrmMetadataModule.cs`, `Metadata.SRM/SrmMetadataProjection.cs`, the `MetadataAuthority*` and `StaticField*Catalog*` families in `Product.DumpQuery`, and `MetadataEditGenerationContracts.cs`. Pair them with `MetadataIdentityTests.cs`, signature projection tests, and the W7/W8 metadata integration suites.

The ClrMD host adapter and physical evidence

7.1 Adapter ownership

`PhoenixInspect.Host.Dump.ClrMD` is the physical boundary between `PhoenixInspect` contracts and `ClrMD`/runtime implementation objects. `ClrmdDumpSession` owns every `ClrMD` lifetime. No `DataTarget`, `ClrRuntime`, `ClrHeap`, `ClrType`, or similar adapter object escapes into product or UI layers. Escaping one would leak thread-affinity, lifetime, caching, and version assumptions into code that currently depends only on immutable `PhoenixInspect` identities.

The session is a large partial class because its surface is grouped by evidence family: arrays, references, frame variables, method/source resolution, static fields, and expression context. The central file owns opening, world identity, module catalogs, memory, and common policy.

7.2 Opening a dump

The open path is deliberately conservative:

1. validate and open the dump file under a maximum file-size policy;
2. keep a read-only stream alive for the session so replacing the path cannot change bytes behind an established identity;
3. hash the complete dump for snapshot identity;
4. create the `ClrMD` data target using an offline component locator rather than ambient machine resolution;
5. require the admitted runtime shape (currently one CLR runtime for the main walking-skeleton path);
6. create a counted process-memory reader with the target's pointer width and stable memory-source identity;
7. enumerate bounded runtime module instances into immutable `PhoenixInspect` descriptors;

8. return an exact session or a typed unavailable/unsupported/invalid result.

The dump cache is bounded and selected ClrMD caches are disabled where they could blur counted observation. The adapter is not promising arbitrary cross-version runtime layout support; the tested surface is tied to named fixtures and same-toolchain evidence.

7.3 Live attach

`AttachToProcess` suspends the target for the lifetime of the session. Disposal detaches and resumes it. The session records process identity and attach time, but explicitly states that its digest is an attach-circumstances identity rather than an immutable content hash.

This lets the same inspection APIs work against dump and live sources while preserving the semantic difference. A host report explains that a live read is repeatable only while this suspended session exists.

7.4 Counted raw memory

`IProcessMemoryReader` and `MemoryReadResult` distinguish:

- exact read: every requested byte was obtained;
- short/partial read: a prefix was obtained and its exact length is known;
- unavailable read: no admissible bytes were obtained;
- invalid request: range, width, or structural request was malformed.

A caller cannot decode an `Int32` or pointer unless the required byte width was read exactly. The missing suffix is never zero-filled. Each read is bounded and carries source identity so result provenance can name the physical memory source and address.

Invariant

“Address exists” and “value is readable” are separate facts. A runtime structure can identify a field location while a sparse dump omits the page containing its value. PhoenixInspect retains the exact location evidence and reports the storage read as partial or unavailable.

7.5 Module instances

A runtime can load logically identical module content into multiple application domains or contexts. The adapter therefore exposes module *instances* with runtime identity, application-domain identity, metadata address/length, layout, and a target-side path hint. Product binding first selects a metadata declaration, then requires exactly one matching runtime module instance under the active contract. Zero matches are unavailable; multiple matches are ambiguous.

Module content identity is computed from counted metadata bytes read from the snapshot, not from the hinted disk path. The hint can seed PDB or assembly candidate discovery, but each candidate must independently reproduce the expected identity.

7.6 Heap, roots, and object identity

The current heap-search surface is deliberately rooted. A bounded strong-handle traversal searches exact ordinal type names and returns retained matches plus traversal counters and caps. This avoids pretending that a filtered or partial heap walk found every object.

An exact result can be converted to a `DumpObjectBinding`. Static-field evaluation can also promote an exact non-null object reference into a root. In both cases the binding records snapshot, object address/type identity, and provenance describing how the root was selected.

Object addresses are never globally meaningful. The same numeric address in another snapshot does not identify the same object, and the query engine checks this before binding a field.

7.7 Instance fields and arrays

Field lookup combines runtime type information with declared field identity and bounded ancestry traversal. The returned descriptor carries offset/type/declaration facts needed for later reads. A read then validates object/reference shape, computes address geometry under target pointer width, obtains exact bytes or a typed short/unavailable result, and decodes only the admitted value kinds.

Arrays are represented as immutable virtual sequences when their shape and element reads can be established. The constant evaluator can consume those sequences through deterministic operators without copying target objects into the analysis runtime's ordinary collection semantics. Bounds are retained for dimensions, lengths, element reads, and result materialization.

7.8 Frames and variables

A selected frame is named by snapshot-scoped thread and frame ordinals. The adapter intentionally does not publish a fabricated thread count. Probing an ordinal beyond the end and probing a live thread with no managed frame can produce the same typed unavailable observation; the inspection service explains this ambiguity instead of inferring which one occurred.

Exact managed frames retain runtime thread identity, module/method tokens, stack/instruction context, and module content identity. Variable projection decodes parameters and local slots where the dump/runtime/PDB evidence permits. Optimized code can legitimately make source locals unavailable or remapped; separate optimized-context fixtures preserve those limits as expected behavior.

7.9 Method bodies and execution resolver

`ClrmdDumpExecutionResolver` joins runtime method observations, counted module metadata, and the reusable SRM projection into execution-core definitions. It is responsible for the host-specific part of W3/W4 admission: locating the exact runtime/module method, reading the admitted body bytes, resolving same-module operands, and exposing structured failures.

The execution core never sees ClrMD methods. It sees `ResolvedMethodDefinition`, `ResolvedField`, and structural handles. This keeps graph planning deterministic and testable with synthetic resolvers.

7.10 Static fields

Static storage is one of the adapter's densest areas because CLR storage varies by field kind, declaring type, app domain, thread/context, nullable layout, and runtime implementation. The W7/W8 contracts split the path into:

1. metadata declaration identity;
2. runtime module/type/field mapping;
3. decoder expectation;
4. storage-base/location evidence;
5. counted raw read;
6. terminal value observation;
7. optional object-reference and assignability proof.

A nullable `Int32`, for example, requires an independently projected runtime layout for its `HasValue` and `Value` children. The product composes metadata child roles with the runtime layout and stops if they conflict.

7.11 Portable PDB and source location

The adapter validates PDB candidates against module debug identity and returns a source *location*: document identity, checksum algorithm/checksum, line span, and artifact identity. It does not read and display the local source file. That final trust decision belongs to the inspection layer because it may involve local files, embedded source, explicit `SourceLink` network capability, or decompilation presentation.

This separation allows headless product tests to prove binding/context facts without granting arbitrary file or network access.

7.12 Session-level bounds

Representative caps include:

- dump file size up to 8 GiB;
- bounded cache memory;

- 4,096 module instances and 4,096 instance-field traversal units;
- 100,000 handle scans and 4,096 retained handle matches;
- 10,000 method scans for relevant host operations;
- 1,048,576 observed string characters at the broad adapter boundary, with tighter product-specific limits;
- bounded frame/thread traversal, PDB candidates, artifact bytes, and raw read widths.

A product layer can impose a stricter bound than the adapter. Both can appear in evidence when both were reached; configured but irrelevant caps do not.

7.13 The dedicated adapter thread

ClrMD's underlying reader is not treated as concurrently safe. `DumpSessionHost` in the inspection project owns one long-lived worker thread and a `BlockingCollection<Action>`. Every session open, close, attach, and query runs on that thread; results are marshalled back through `TaskCompletionSource<T>` with asynchronous continuations.

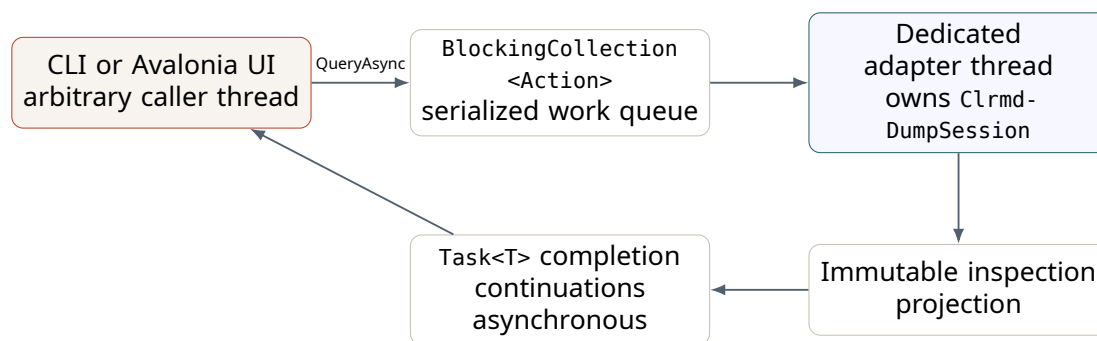


Figure 7.1: Host-side serialization of every ClrMD operation. Only immutable PhoenixInspect projections return to UI code.

Opening another world disposes the prior session first. This prevents a rendered result from mixing module lists or object roots from two snapshots. Disposing the host closes the session on the worker thread, completes the queue, and joins the worker.

7.14 Where adapter bugs usually hide

- a ClrMD object escaped its session or thread;
- a target-side path hint was accidentally treated as identity;
- a short read was decoded as a full scalar;
- a runtime module instance join ignored app-domain/runtime identity;
- a bound was reached but not carried into the product result;
- a cached runtime observation came from a different world;
- a runtime layout assumption was not pinned to tested descriptor/toolchain evidence;
- a filtered dump was treated as “zero” rather than unavailable.

Primary code landmarks

Begin with `ClrmdDumpSession.cs`, `ClrmdProcessMemoryReader.cs`, `ClrmdModuleInfo.cs`, and `SnapshotIdentity.cs`; then follow the relevant partial file for arrays, references, frames, source, or static fields. For execution, read `ClrmdDumpExecutionResolver.cs`. For threading, read `Inspection/Infrastructure/DumpSessionHost.cs`. Real-dump behavior is concentrated in `PhoenixInspect.IntegrationTests`.

Parsing, binding, rooted queries, and constant evaluation

8.1 Why `Product.DumpQuery` is larger than its name

The project began with a narrow root-field query and now contains several cooperating expression subsystems:

- the sole pinned Roslyn C# expression parser and versioned admission profiles;
- the original W2 rooted field query and immutable plan;
- fixed-depth and later arbitrary-depth admitted member chains;
- W7/W8 static-field syntax, metadata binding, runtime mapping, and storage reads;
- a large deterministic constant-expression evaluator;
- type/enum/array/sequence/temporal/query/anonymous-object value models;
- source-anchored metadata authority and edit-generation contracts.

Its unifying principle is read-only derived truth. Even when it evaluates rich syntax, it either folds deterministic values or composes observations from the snapshot. It does not interpret arbitrary user methods; that transition belongs to `Product.DumpDebugging`.

8.2 One pinned parser

`CSharpExpressionFrontEnd` and `DumpQueryParser` centralize parsing under Roslyn 5.3.0, C# 14, and a named project profile. The parser accepts a complete expression with full-text consumption and rejects directives, disabled text, skipped tokens, missing tokens, overlong input, excessive syntax depth, or excessive node/token count.

Representative bounds include:

- 512 expression characters;
- 256 syntax nodes plus tokens;
- syntax depth 64;
- identifier length 64;

- bounded string-literal length.

The front end distinguishes invalid C# from valid C# outside the current project-owned profile. This is essential for forward evolution: adding an admitted construct changes the profile version and contract, not the meaning of “valid syntax.”

8.3 Versioned expression descriptors

After Roslyn parsing, the product converts syntax to immutable project-owned descriptors. Downstream code does not retain Roslyn nodes as semantic identities. Profiles include historical frozen W2/W5 forms, fixed-depth member-chain admission, and the current opt-in member-chain V2 surface.

The descriptor records normalized structural information but preserves enough raw/request identity to produce stable policy provenance. Selection occurs once. Later stages use the descriptor and selected plan rather than reparsing or rediscovering candidate shapes.

Invariant

Roslyn answers “what syntax did the user write?” PhoenixInspect answers “which subset and semantic profile is admitted?” The second question must never be delegated to whatever Roslyn happens to support in a future package version.

8.4 The W2 rooted query

DumpQueryEngine remains a useful miniature of the whole architecture. Its original grammar is intentionally tiny: one exact root identifier, one instance-field name, and optional null/Int32/string coalescing.

8.4.1 Preparation

Preparation performs:

1. syntax classification under parser bounds;
2. validation of upstream root-binding bounds;
3. snapshot identity comparison;
4. root-binding status check;
5. one bounded runtime field selection;
6. field-kind classification and coalesce compatibility;
7. construction of an immutable DumpQueryPlan carrying the selected root, field descriptor, parser/adaptor bounds, and provenance identity.

A search-backed root retains the exact type predicate, adapter status, traversal counters, and caps. This means two roots at the same address selected under different evidence do not automatically yield the same plan identity.

8.4.2 Evaluation

Evaluation consumes the frozen field descriptor. It does not call member lookup again. It reads the selected field under the exact snapshot and decodes according to the plan's field kind. Exact null, exact Int32, bounded string, partial bytes, and unavailable bytes remain distinct.

This separation is small enough to study but general enough to serve as a pattern for later member chains and methods.

8.5 Member chains

The member-chain engine extends the same discipline across multiple hops. The syntax descriptor determines the exact chain, conditional-access boundaries, and fallback. Each hop:

1. starts from an exact object or nullable/reference state produced by the prior hop;
2. selects a declared runtime field once;
3. records its field identity and evidence;
4. reads and decodes the value under bounds;
5. either continues with an object binding, returns a terminal value, applies conditional-null semantics, or stops with a typed disposition.

An arbitrary-depth user expression is therefore still a finite bounded operation: expression length and node count bound the number of admitted segments; each field/ancestry/read operation has its own cap; result materialization has another cap.

Conditional access is semantic, not an exception suppressor. It only bypasses the suffix when the receiver is an exact null under the admitted nullable/reference semantics. An unavailable receiver does not become null.

8.6 Static-field expression pipeline

StaticFieldExpressionEvaluator runs a larger parse/bind/map/read/compose pipeline. Its high-level stages are:

Stage	What is established
Syntax	Exactly one accepted static-field descriptor and candidate-shape set.
Symbol binding	One metadata declaration, or a typed absent/ambiguous/context-required/invalid stop.
Runtime declaration	Exactly one runtime module/type/field mapping corresponding to that metadata declaration.
Nullable layout	Exact runtime child-role geometry when the declared type is nullable Int32.
Storage request	A physical read request whose decoder and storage identity agree with metadata/runtime evidence.

Stage	What is established
Storage observation	Counted bytes and terminal scalar/reference/null outcome.
Assignability	For non-null references, proof that the runtime object is admissible for the declared field type.
Suffix evaluation	Optional direct member/chain/fallback evaluation over an exact object-reference result.
Composition	One replayable result retaining exact prefixes, stops, bounds, observations, and optional promotable root.

8.6.1 Lazy context acquisition

The overload taking a selected-frame selector and PDB candidates does not immediately inspect the frame. It first tries context-independent fully qualified binding. Only when a bare/source-like spelling genuinely needs imports, namespace, or aliases does it acquire frame/PDB context.

This avoids three problems:

- a fully qualified expression should not fail because an unrelated frame is unavailable;
- exact context-independent replay should not depend on a PDB candidate set it never consulted;
- the adapter should not perform expensive or capability-sensitive work speculatively.

A descriptor containing a global qualifier also remains context-independent by construction.

8.6.2 Metadata-to-runtime join

After exact symbol binding, the evaluator matches the selected declaration's module identity against runtime module instances. Exactly one match is required. It then calls the session's runtime-declaration mapper using definition tokens, declaring-type identity, field name, and expected decoder. These redundant-looking inputs cross-check one another; they are not merely lookup conveniences.

Nullable layout and reference assignability are composed in separate steps so a failure can name the exact unsupported/conflicting boundary.

8.6.3 Root promotion

An exact non-null object-reference static field can produce a `DumpObjectBinding` with provenance identifying the static expression. The inspection layer exposes this as a promotable root. This is necessary because an object can be reachable from a static field without appearing in the bounded strong-handle catalog.

8.7 The deterministic constant evaluator

`ConstantExpressionEvaluator` is split across partial files by semantic family: numeric, syntax, arrays, sequences, queries, lambdas, temporal values, enums, types, tuples, anonymous objects, and supporting value models. It is best understood as a pure interpreter for a closed expression language, not as “compile and invoke C#.”

Its top-level status is one of:

NotConstant

The expression should fall through to another product pipeline unchanged.

Exact

A deterministic value was produced under the admitted constant/profile semantics.

Invalid

The expression selected this pipeline but violated syntax, typing, bounds, or deterministic operation rules.

A bare dump field or member chain deliberately remains outside the constant pre-stage so its existing evidence report and replay digest stay byte-identical. A composed expression such as `root.QueueDepth * 2 + 1` enters the constant evaluator, which calls the frozen root pipeline for the operand and consumes its exact value.

8.8 Value families

The evaluator’s result model includes `Int32`, general numeric values, strings/chars/booleans/null, enums, sequences, temporal values, deterministic BCL values, type references, tuples, and anonymous objects. The implementation aims to reproduce C# and selected BCL semantics rather than invent convenient approximations.

8.8.1 Numeric semantics

The supported tower includes fixed-size integral types, `Int128/UInt128`, `BigInteger`, IEEE-754 float/double, and exact-scale decimal. Promotions, checked/unchecked contexts, conversions, comparisons, and formatting are explicit. Host culture is not consulted.

8.8.2 Sequences and LINQ

Array initializers, collection expressions with spreads, dump-backed arrays, and folded sequences share a virtual sequence model. Deterministic `Enumerable` operations and expression lambdas execute within bounded evaluator semantics. Operations that mutate or rely on ambient comparer/culture/time state are rejected or modelled only under an explicit deterministic contract.

8.8.3 Temporal and BCL values

`DateTime`, `DateTimeOffset`, `TimeSpan`, `DateOnly`, and `TimeOnly` are admitted for deterministic construction, ticks, arithmetic, comparison, and invariant formatting. `Now`, local-time

conversion, and culture-dependent parsing are typed stops. Likewise, `Guid` and `Version` have fixed-grammar construction/parsing and comparison; `Guid.NewGuid()` is not evidence and is rejected.

8.8.4 Enums and types

Enum shape is read from the captured module's metadata. Casts, bitwise flags, `HasFlag`, formatting, and selected `System.Enum` operations use that target authority. `typeof` and deterministic `System.Type` operations operate over structural type values, including arrays, generic definitions/instantiations, assignability, variance, and declared base/interface relationships where exact metadata support exists.

8.8.5 Queries and modern C# forms

Query syntax is translated according to C# rules onto the admitted folded operator surface. Anonymous objects retain projected names, structural equality, invariant formatting, and expandable members. Interpolated strings, selected patterns and switch expressions, `nameof`, `default(T)`, `sizeof(T)`, and checked wrappers are handled within the same deterministic restrictions.

8.9 Canonical identity and display expansion

The constant outcome has its own domain/versioned canonical digest. Expandable children for arrays, tuples, and anonymous objects are a presentation projection and do not perturb the value digest. This lets a host materialize a bounded child list without changing the identity of the parent answer.

8.10 Expression routing above the project

The shared inspection layer routes a `Watch` expression lexically:

- if an adopted root exists and the root identifier appears as a standalone token, use the root-relative product path;
- otherwise use the static-field path.

The rule is deliberately total and non-parsing. The root-relative path can also resolve context-free static operands, so a mixed expression can route there and combine both. An occurrence inside an interpolated string still routes to the root path; at worst, the richer resolver answers an expression the static path could also answer.

8.11 Failure interpretation

When debugging an expression result, identify the last completed stage:

1. Roslyn syntax;

2. project profile admission;
3. metadata binding;
4. runtime declaration mapping;
5. physical storage or object hop;
6. deterministic value composition;
7. method-path handoff, if any.

A failure at stage 3 should not be “fixed” by making storage lookup more permissive. The typed result and retained exact prefix exist precisely to prevent such cross-layer guessing.

Primary code landmarks

Core reading order: `DumpQueryParser.cs`; `DumpQueryEngine.cs` and `DumpQueryPlan.cs`; member-chain `parser/plan/engine` files; `StaticFieldExpressionEvaluator.cs`; then the partial `ConstantExpressionEvaluator.*.cs` families. Tests are organized similarly: `parser/plan` tests, `ConstantExpression*Tests.cs`, and `real-dump member/static-field` suites.

Counterfactual method evaluation

9.1 The product-level handoff

`PhoenixInspect.Product.DumpDebugging` is where dump evidence becomes a counterfactual machine input. It owns expression classification, method acquisition, virtual memory import, request/preparation contracts, execution, and result projection. It references the query product because field/member/static operands can be part of the expression path and because their exact evidence is reused rather than reimplemented.

`DumpExpressionEvaluator` returns a strict union. Representative alternatives include:

- derived query result;
- counterfactual execution result;
- classification failure;
- dump acquisition failure;
- counterfactual preparation failure.

The facade does not force every route into one fake lattice. A W2 field query and a W4 method execution retain their native contracts and semantic modes.

9.2 Classification before evidence

Expression classification occurs before touching dump evidence. Rejected syntax cannot enumerate modules, read fields, or query resolvers. This capability ordering is tested and matters for both performance and replay: a syntax-only rejection should not depend on which dump happened to be open.

For a method path, classification yields a structural descriptor that drives acquisition. The product then resolves the rooted receiver, method declaration, arguments, field observations, and policy into a preparation candidate.

9.3 Counterfactual request identity

`CounterfactualMethodRequest` freezes the public semantic request:

- evidence source and snapshot/module identity;
- root-selection identity and root evidence digest;
- root method and receiver/argument evidence;
- policy ID/version and instruction/depth/traversal limits;
- model catalog ID/version and optional required modeled target;
- explicit assumptions.

The request is validated for mutual compatibility before graph planning. A malformed candidate becomes a stable value-free failure, not an exception-controlled result.

9.4 Preparation candidate and private runtime bundle

The preparation candidate additionally carries capabilities and data that should not become public request identity: resolver, value domain, memory model, initial persistent memory, materialized receiver/arguments, field observations, and optional model registry.

After validation and graph preparation, these are sealed into a private runtime bundle. The public plan exposes canonical facts; the executable capabilities remain private. This protects replay from callers swapping a resolver or memory model after a plan's digest has been computed.

9.5 Private plan issuance

Each `CounterfactualMethodRunner<TMemory>` creates a private issuer object. Successful plans are stamped with that issuer. Run checks plan authority before observing cancellation, reading public plan fields, or touching private capabilities. A null or foreign plan returns a fixed identity-free facade rejection.

This is stronger than a type check. A plan issued by another runner instance is structurally plausible but not authorized to use this runner's retained runtime bundle. The design resembles an object-capability boundary inside the process.

9.6 Preparation sequence

Prepare performs, in order:

1. null/raw candidate validation;
2. canonical request creation;
3. capability/binding compatibility checks;
4. one graph-planning attempt, with optional required pure model;
5. planner success/failure coherence validation;
6. logical-depth sufficiency check;

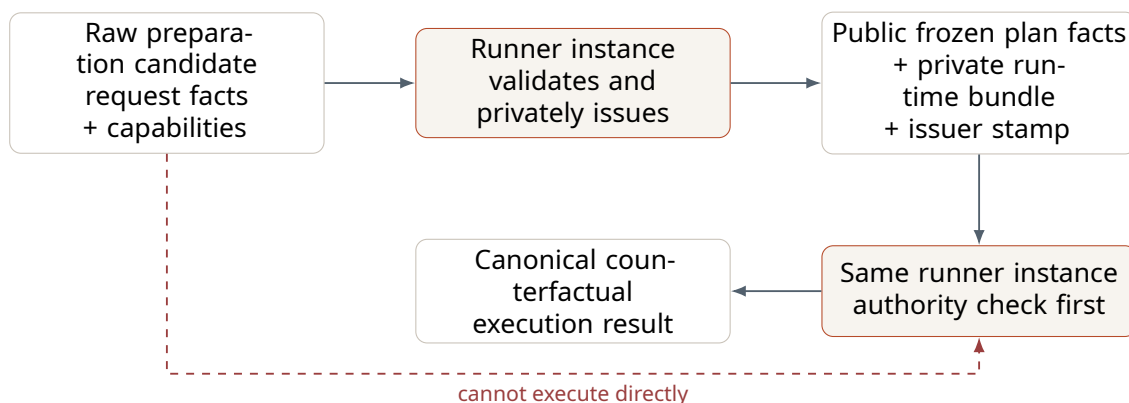


Figure 9.1: Private issuance prevents a caller from assembling an executable-looking plan from public data.

7. root signature versus receiver/argument compatibility;
8. field-observation completeness/order/receiver validation;
9. provenance-domain argument materialization;
10. private plan issuance.

Every ordinary malformed-input exception is normalized into a stable result. Catastrophic process exceptions are not swallowed.

9.7 Recording memory

Execution wraps the underlying memory model in `CounterfactualRecordingMemoryModel`. The wrapper knows the predeclared field observations and the rooted receiver. It rejects an unexpected load rather than querying ambient dump state during execution. Reached load ordinals and observations are recorded in exact dynamic order.

This closes a subtle replay hole. A frozen graph can prove which fields are syntactically reachable, but branch/model behavior determines which loads actually occur. The result should cite reached evidence, while still ensuring no undeclared load can escape the plan.

9.8 Rejecting execution-time resolution

The machine is created with `RejectingExecutionResolver.Instance`. Any accidental resolution attempt inside the execution loop is therefore a hard capability failure. All definitions and call targets must come from the frozen graph.

This is one of the best architectural tests in the repository: rather than relying on code review to notice a resolver call, the product supplies a resolver whose only valid behavior is rejection.

9.9 The run loop

Before stepping, the runner:

- calculates a finite dynamic instruction cost from the acyclic graph;
- creates provenance domain, recording memory, machine, and operational budget;
- activates the prepared graph;
- validates the exact single-root activation state;
- initializes call trace and event builders;
- derives a maximum step-request count including the terminal idempotence check.

Each iteration then:

1. validates the state envelope;
2. observes cancellation only at a ready boundary;
3. enforces the graph-derived step-request maximum;
4. calls `StepOne`;
5. verifies the machine-issued transition stamp;
6. validates budget, event, frame, model, memory, and terminal-union deltas;
7. appends dynamic calls/events;
8. dispatches on `Ready`, `Completed`, `BudgetExhausted`, `Blocked`, `InvalidProgram`, or target exception.

A capability exception becomes a stable blocked/invalid result depending on where it occurred. An impossible transition becomes a replay-invalid result with a diagnostic naming the violated contract.

9.10 Completion and result projection

A normal completion must leave an empty call stack, one root return value, no terminal target exception, and the same read-only initial memory identity. The mandatory terminal re-step must be idempotent. The runner then projects:

- semantic mode `CounterfactualExecution`;
- completion/completeness/evidence/effects axes;
- typed return or execution-prefix value;
- instruction/depth/lineage bound status and high-water marks;
- reached field observations and load ordinals;
- model attempts and counts;
- dynamic call trace and debug events;
- stable diagnostics and canonical replay identity.

Effect status reflects virtual or modeled behavior, never mutation of the captured process.

9.11 Why the runner revalidates its own machine

At first glance, transition validation appears redundant: the machine and runner live in the same repository. It serves several purposes:

- the generic machine consumes external domain/memory/model capabilities whose implementations can be buggy;
- product-level canonical results need stronger invariants than a low-level stepper's convenience API;

- tests can inject hostile or incoherent capabilities and prove prefix-free failure;
- future machine changes cannot silently alter event/budget/frame contracts without breaking product tests;
- replay identity is protected from malformed internal states rather than trusting them by convention.

The runner is thus a certification boundary, not a thin loop wrapper.

9.12 Modeled calls

A pure model has a versioned catalog identity and explicit attempt result. The operational envelope retains every attempt, whether it completed a transfer or failed. A successful model consumes one instruction transfer under the closed contract, advances logical depth, may avoid an interpreted frame, and contributes transformation provenance.

A required model target is part of the request. The planner cannot silently fall back to interpretation if the required model is absent, nor can an interpreted-only policy consult a model registry.

9.13 Counterfactual memory and assumptions

The method path imports exact root and field observations into virtual memory. Assumptions are explicit request fields, not ambient facts. The product can therefore distinguish:

- an exact captured receiver field;
- a virtual value introduced under a named assumption;
- a modeled helper result;
- an unknown caused by unsupported or missing evidence.

That distinction will be necessary for future virtual stepping or abstract analysis, where the virtual world diverges further from the dump.

Primary code landmarks

Read `DumpExpressionEvaluator.cs`, the `CounterfactualMethodRequest*`, `CounterfactualMethodPreparation*`, `CounterfactualMethodPlan*`, `CounterfactualRecordingMemoryModel.cs`, and `CounterfactualMethodRunner.cs`. The `result/canonical-codec` files explain final replay shape. The corresponding `Counterfactual*Tests.cs` and `PreparedGraph*Tests.cs` cover hostile and edge-case capabilities.

Inspection services and the three hosts

10.1 A deliberately thin presentation layer

`PhoenixInspect.Inspection` is the shared host-facing layer. Both the console and Avalonia applications reference it; neither references the product internals directly. Its job is to:

- own one serialized dump/live session;
- invoke adapter and product entry points on the correct thread;
- project outcomes into immutable display models;
- format stable facts without changing their meaning;
- maintain host-facing root/frame/PDB context;
- provide completion, process discovery, source navigation, and expansion services.

Its XML comments repeatedly state that it does not widen or reinterpret adapter/product outcomes. Statuses, issues, counters, bounds, and digests are carried through. This is the architectural reason the CLI and desktop produce semantically comparable answers.

10.2 Display models

The models project defines value-like records for snapshot properties, module rows, frame variables, call-stack nodes, heap matches, diagnostics, evidence facts, source lines, and evaluation reports. They are deliberately detached from ClrMD lifetimes. A UI may cache or bind them after the adapter operation completes.

`EvaluationReport` is not a replacement result contract. It is a host-oriented projection containing the original status vocabulary, path/stage description, rendered value/type, evidence facts, diagnostics, reached bounds, digest, optional expandable children, and optional promotable root.

10.3 DumpInspectionService

This service handles non-expression inspection:

- snapshot/live-session overview and declared-bound facts;
- module-instance listing and counted metadata identity;
- bounded thread probing and frame loading;
- parameters/locals projection;
- heap-object search;
- source-selection inputs and related rows.

A particularly good example is call-stack probing. The service does not infer a thread count the adapter does not provide. It returns nodes for ordinals with an exact managed frame and an explanatory note for indistinguishable unavailable cases. Presentation text is thus derived from the adapter's epistemic boundary rather than from debugger convention.

10.4 ExpressionEvaluationService

The expression service presents two host-facing paths:

Static field

Fully qualified or contextual static expressions, with the constant pre-stage and optional root promotion.

Root relative

Member/method/composed expressions against an adopted root under host-friendly policy options.

Host options are mapped to product contracts: modeled versus interpreted methods, member-chain profile, instruction limit, logical-depth limit, and traversal limit. The service clamps values to product maxima so a front end cannot accidentally bypass hard policy.

Watch routing uses the standalone-token scan described in Chapter 8. The report always names the path that answered. This makes a transcript auditable even if the same text could in principle be resolved by more than one subsystem.

10.5 Completion

Completion candidates are evidence-derived and profile-aware. The completion service combines:

- C# keywords and modeled deterministic types/members;
- root fields from the adopted object's exact runtime type;
- namespaces, types, and static fields from dump-module metadata;
- contextual/import information when exact frame/PDB context is active.

The intended invariant is that completion should not suggest a name the evaluator's own current profile cannot answer. Candidate enumeration is bounded and carries session state so a stale drop-down does not become authority after root/context changes.

10.6 Source navigation

SourceNavigationService deliberately separates line mapping from content trust. The adapter establishes which document and span the matching PDB recorded. The inspection service then tries content sources in an evidence-ranked order:

1. local file at the recorded path, only if its bytes reproduce the PDB document checksum;
2. embedded source inside the identity-matching Portable PDB;
3. SourceLink content, only when the host explicitly enables download, the URL is HTTPS, the byte cap is respected, and the downloaded bytes reproduce the PDB checksum;
4. C# decompiled from an on-disk assembly whose complete metadata identity matches the dump module, explicitly labelled as reconstruction;
5. otherwise a typed unresolved source view.

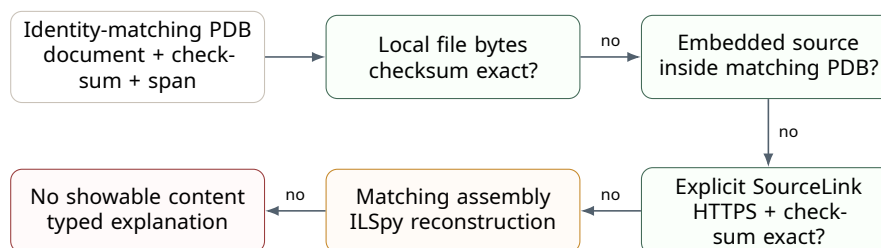


Figure 10.1: Source content is shown only through a proved identity/checksum path; decompiled C# is a separately labelled reconstruction.

The service caps source files at 10 MiB and PDB candidates at 64. Unknown checksum algorithms do not justify a false exact claim. Embedded source can still be shown as artifact-contained evidence with a precise label when no supported checksum algorithm is available, provided the matching PDB itself is authoritative.

10.7 Process discovery

The desktop's Processes pane uses a bounded discovery service to identify likely managed processes and explain the recognition evidence. Discovery does not attach. The user-selected process is then passed to the same session host that the CLI uses, and the attach result still must be exact.

10.8 The console host

PhoenixInspect.Cli is a full scriptable inspection shell rather than a thin one-command wrapper.

10.8.1 Entry and exit codes

Program.Main supports dump capture, dump open, live attach, scripted commands, repeated evaluations, and an interactive prompt. Exit codes distinguish success, usage

error, world-open failure, and command failure. UTF-8 output is selected explicitly.

A session begins by opening/attaching through `DumpSessionHost`, printing the overview, then running either the scripted sequence or interactive loop. The final summary reports total expressions and the count that were not exact or exhaustively absent. Its wording deliberately describes non-exact results as evidence limits rather than failures to “try harder.”

10.8.2 Session commands

`InspectionSession` maintains explicit state:

- adopted root and root identifier;
- selected frame for name context;
- offered Portable-PDB candidates;
- latest heap search and module list;
- verbose-evidence setting and evaluation counts.

Commands include overview/status, modules and module metadata, threads/frames, context selection by ordinal or method name, PDB candidate management/auto-discovery, bounded object search, root adoption from search or static expression, root renaming, evaluation, and exit. A bare line not recognized as a command is evaluated as an expression.

The `status` command makes every evidence input visible. This is important for scripted transcripts: a reader can reconstruct why `ServiceState.Count` bound contextually in one step and required qualification in another.

A command-level exception is rendered and returns a failed command outcome without destroying the open session. Ordinary product non-exact results are not exceptions and do not fail a script merely because they are partial or unsupported.

10.9 The desktop host

`PhoenixInspect.Desktop` uses `Avalonia`, `Dock`, `AvaloniaEdit`, and `ProDataGrid` to present a debugger-style workspace. The source tree separates:

- `ViewModels/`: main window, session overview, processes, modules, threads, call stacks, locals, heap objects, Evaluate, Watch, and Immediate;
- `Views/`: pane XAML and small view-specific interaction code;
- `Docking/`: dock factory and pane models;
- `Infrastructure/` and converters: UI-only support;
- application resources/themes in `App.axaml`.

The default layout resembles a debugger: navigation/inspection panes, central documents, evidence/result pane, and bottom-band evaluation/call-stack tools. Docking is a presentation concern. View models call the same inspection services as the CLI.

10.9.1 Watch

Watch rows are editable. The trailing row creates a new watch; clearing a row deletes it. Rows re-evaluate when root or frame context changes. Compound values expose bounded child rows from the report's expansion projection. Each row retains its own status and evidence report rather than sharing a global "last error."

10.9.2 Immediate

The Immediate pane is a prompt/transcript over the same routed evaluator. History navigation and completion are UI state; expression semantics are not reimplemented. Results are rendered with C#-like syntax colouring and trailing status comments, and explained stops are comments rather than bare exception text.

10.9.3 Source documents

Selecting a frame can open a verified source or reconstructed decompilation document. The document title and summary state which evidence path produced it. This avoids a common debugger UI problem where a stale local source file is displayed with authoritative highlighting.

10.10 The headless reference consumer

`PhoenixInspect.Headless.ReferenceConsumer` is both a sample client of the public APIs and an executable conformance/corpus host. Its large portfolio runners predeclare scenarios, invoke exact public entry points, render stable reports, compare canonical digests/goldens, and prove that useful analysis does not depend on desktop types.

This project is strategically important:

- it catches accidental internal-only dependencies in public contracts;
- it supplies a deterministic hosted CI workflow;
- it records meaningful synthetic incidents and counterfactual variants;
- it provides closure evidence for milestone claims;
- it demonstrates how another host could integrate the libraries without copying UI code.

10.11 Presentation-layer anti-patterns

- Calling `ClrMD` directly from a view model or CLI command.
- Rebinding or reparsing an expression to obtain nicer display text.
- Treating `Partial` as a red exception and `Absent` as an empty string.
- Caching a root/frame across session replacement without snapshot identity.
- Showing a local source file before checksum verification.
- Inventing a UI-only diagnostic code for a product semantic stop.
- Letting desktop and CLI assemble different method policies for the same visible option.

Primary code landmarks

Shared host files: `Inspection/Infrastructure/DumpSessionHost.cs`, `Inspection/Services/DumpInspectionService.cs`, `ExpressionEvaluationService.cs`, `ExpressionCompletionService.cs`, and `SourceNavigationService.cs`. Console flow begins at `Cli/Program.cs` and `Cli/InspectionSession.cs`. Desktop composition begins at `Desktop/ViewModels/MainWindowViewModel.cs` and `Desktop/Docking/InspectionDockFactory.cs`. The headless portfolios are intentionally large and worth reading after the core APIs.

End-to-end request walkthroughs

This chapter traces representative user actions across project boundaries. The precise internal types vary by expression shape, but these paths are stable mental models for debugging.

11.1 Opening a dump

11.1.1 User action

The user starts the CLI with a dump path or chooses Open in the desktop shell.

11.1.2 Control flow

1. The host calls `DumpSessionHost.OpenAsync`.
2. Work is queued to the dedicated adapter thread.
3. Any prior session is disposed; its roots, frames, and modules become unreachable.
4. File length is acquired for display; `ClrmdDumpSession.Open` performs the authoritative open.
5. The session hashes the complete dump, opens ClrMD with the offline locator, creates memory/module catalogs, and returns a typed adapter result.
6. Only an exact result is installed as the active session.
7. `DumpInspectionService.LoadSnapshot` projects snapshot identity, target platform/architecture, module rows, declared bounds, and expression profile.
8. The immutable projection returns to the UI thread and is rendered.

11.1.3 Key invariants

No object from the previous session survives as an admissible root. The displayed path is not snapshot identity. A non-exact open leaves the host closed. Module rows are instances, not deduplicated simple names.

11.2 Evaluating a fully qualified static field

Consider:

```
Contoso.OrderService.Diagnostics.ServiceState.ProcessedOrderCount
```

1. The CLI/Watch calls `ExpressionEvaluationService.EvaluateStaticField` on the adapter thread.
2. The constant pre-stage asks whether the entire expression is a foldable composition. A bare stored field returns `NotConstant` and falls through unchanged.
3. `StaticFieldExpressionParser` parses once and produces candidate shapes.
4. The fully qualified binder consults the metadata authority. No frame or PDB is touched.
5. Exact symbol binding selects module/type/field definition identities.
6. The evaluator joins that module identity to exactly one runtime module instance.
7. `ClrmdDumpSession.MapStaticFieldDeclaration` proves the runtime field mapping and decoder shape.
8. The product creates a physical read request and the session reads storage through counted memory.
9. The host observation is composed with metadata declaration semantics into a `StaticFieldObservation`.
10. `ExpressionEvaluationService` projects value, type, stage, status, raw reads, binding facts, reached bounds, diagnostics, and replay digest.
11. The host renders `Exact/Complete` only if every required stage was exact.

If the field is an object reference, the report may carry a promotable `RootSelection`; promotion is not a second heap search.

11.3 Using a selected frame for contextual binding

Consider a source-like name that omits namespace qualification.

1. The session's explicit context points to a snapshot-scoped thread/frame selector and a bounded PDB candidate list.
2. Static-field syntax is parsed first.
3. If a context-independent route can bind exactly, context acquisition is skipped.
4. Otherwise the session reselects the exact frame, validates module/method identity, validates a PDB artifact, and projects namespace/import/alias facts.
5. The contextual binder tries source-legal candidate readings under the lexical envelope.
6. Complete versus incomplete scope evidence controls whether a miss can become exhaustive absence.
7. The selected declaration then follows the same runtime mapping and storage path as the fully qualified case.

A stale PDB can neither contribute imports nor source. A missing PDB produces a typed context limitation, not a fallback to the analysis machine's project references.

11.4 Adopting a root and walking a chain

Suppose a static expression produces the dispatcher object, which the user adopts as root, then evaluates:

```
root.CurrentBatch.Route.Corridor.Name
```

1. Watch routing detects the standalone root identifier and selects the root-relative path.
2. `RootSelection.Resolve` validates that the root belongs to the open snapshot and re-constructs the exact product binding.
3. The constant pre-stage sees a bare chain and returns `NotConstant`, preserving the member-chain report.
4. The member-chain profile admits the parsed segments.
5. For `CurrentBatch`, the engine selects one exact runtime field, reads it, validates a non-null object reference, and creates the next object binding.
6. The same sequence repeats for `Route` and `Corridor`.
7. `Name` is read as a bounded target string; exact length/content require exact object/string bytes under the relevant cap.
8. The final report aggregates hop identities, raw reads, bounds, diagnostics, and canonical plan/result identity.

A null at an ordinary dot is not the same as unavailable evidence. Conditional access only short-circuits on exact null. A short read at hop three yields a partial/stopped answer retaining the exact first two hops.

11.5 Composing dump values with constants

For:

```
root.QueueDepth * 2 + 1
```

1. Root routing selects the root-relative path.
2. The constant pre-stage recognizes a composed expression rather than a bare chain.
3. Its operand resolver invokes the frozen root-chain evaluator for `root.QueueDepth`.
4. Only an exact admissible scalar becomes a constant operand; non-exact evidence is projected into a typed composition stop.
5. Numeric promotion and arithmetic run in the deterministic constant semantics.
6. The result's provenance includes the dump-derived operand and the transformation policy; the semantic mode remains a derived query.

The evaluator does not substitute a display string or reflection value. It consumes the exact typed operand supplied by the product pipeline.

11.6 Evaluating an admitted method

For a rooted property/helper expression whose implementation requires user IL:

1. `DumpExpressionEvaluator` classifies the expression as the method path before evidence acquisition.
2. The dump adapter resolves receiver type/method identity, method body, signatures, and declared field observations.
3. Product code imports exact root/field values into a persistent virtual memory and provenance domain.
4. A `CounterfactualMethodPreparationCandidate` is assembled with request identity, policy, resolver, domain, memory, model registry, and observations.
5. The runner validates and freezes the complete call graph exactly once.
6. If depth, signatures, observations, and capabilities agree, the runner privately issues a plan.
7. Run verifies plan authority, activates the graph, and repeatedly requests certified machine transitions.
8. Reached field observations are recorded; calls and events form a dynamic trace; instruction/depth/model accounting is updated.
9. Normal completion is terminally re-stepped for idempotence.
10. The product returns a counterfactual result carrying value, virtual/model effects, evidence, trace, bounds, provenance, diagnostics, and replay digest.

The UI should describe this as interpreted/modeled evaluation, never as a field read or historical observation.

11.7 Opening source from a frame

1. The user selects an exact frame node whose selector can reproduce the frame.
2. The adapter validates candidate PDB identity and maps the frame to a document/span.
3. `SourceNavigationService` tries the recorded local path and hashes bytes with the recorded SHA-1/SHA-256 algorithm.
4. On mismatch/missing content, it tries embedded source, then optional `SourceLink` under explicit capability and checksum.
5. If no original source is provable, it locates the hinted assembly only as a candidate, recomputes metadata identity against the dump module, and uses `ILSpy` to reconstruct the declaring type/method.
6. The resulting document is labelled verified original, matching-PDB embedded/`SourceLink`, decompiled reconstruction, or unresolved.

The line highlight is authoritative only when the content path is authoritative. A decompiled document can point near the method, but it is not assigned the original PDB line span as though the text were identical.

11.8 Reading a non-exact report

A systematic reading order is:

1. **Path and semantic mode:** which product pipeline answered?
2. **Completion:** did it finish, block, exhaust a budget, cancel, or reject invalid state?
3. **Completeness:** is there a full value, a prefix, or no semantic payload?

4. **Evidence:** exact, partial, unavailable, conflict, or invalid?
5. **Stage:** what exact prefix was established before the stop?
6. **Diagnostics:** which named disposition explains it?
7. **Reached bounds:** which cap materially affected the result?
8. **Facts/raw reads/provenance:** what physical and transformational chain supports the claim?
9. **Replay digest:** can this exact contract be compared with another run?

A host that follows this order can be useful even when many real-world dump questions are not exact. The product's central promise is not universal answerability; it is honest, composable answer shape.

Tests, fixtures, corpus evidence, and CI

12.1 Tests are part of the architecture

PhoenixInspect's strongest design claims are expressed twice: in immutable contracts and in a layered evidence suite. Tests do more than check output values. They check capability non-use, ordering, canonical bytes, exact versus partial absence, path independence, transition certification, provenance retention, and real runtime behavior.

The repository uses several complementary forms of proof.

12.2 Unit and model tests

`PhoenixInspect.Tests` exercises code that can be tested without capturing a real dump. Major families include:

- canonical replay and evaluation-result coherence;
- bounded signature decoding and malformed metadata shapes;
- concrete-domain lattice laws and persistent memory;
- IL-machine transfer behavior, activation, budget, events, terminal states, and unknown policy;
- method-graph traversal, caching, canonical order, model selection, and accounting;
- prepared-graph execution and hostile capability results;
- counterfactual request/preparation/runner/result/canonical contracts;
- provenance field/call/model lineage;
- constant expression semantics by value family;
- metadata identity, authority, and edit-generation lineage.

Synthetic resolvers and memory models are especially valuable here. They can count calls, throw at precise boundaries, return malformed unions, or provide contradictory identities. This proves the product's prefix-free handling more thoroughly than a happy real-dump scenario could.

12.3 Compiler differential tests

Where C#/CLI semantics are subtle, tests compare PhoenixInspect's structural projection or evaluator behavior with compiler-produced artifacts. Examples include compiler-emitted signatures, Roslyn query/lambda translation shapes, enum constants, and Edit-and-Continue deltas produced by `EmitDifference`.

The principle is “compiler truth before hand-authored bytes.” A hand-built metadata blob is useful for poison tests but should not define the expected shape of ordinary compiler output.

12.4 Integration tests over real dumps

`PhoenixInspect.IntegrationTests` captures and reopens actual .NET processes. Its test names form a map of supported physical behavior:

- `ClrMD` raw memory, method bodies, resolver graphs, and evaluation projection;
- dump memory evidence, sparse dumps, and foreign-snapshot isolation;
- query parser/plan/corpus behavior against captured fields;
- frame selection, variables, source/PDB identity, local/embedded/decompiled source;
- live attach and managed process discovery;
- W3 getter execution, W4 call lineage, W5/W6/W7/W8 expression paths;
- preview-demo transcript;
- Edit-and-Continue physical truth, fixture prerequisites, refusal, and delta acquisition.

A real dump is indispensable because `ClrMD` surface behavior, runtime storage geometry, sparse-memory coverage, Portable-PDB reachability, and optimization effects cannot be established by mocks.

12.5 Fixture targets

Fixtures are deliberately narrow and often named for one evidence question:

- `PhoenixInspect.TestTarget`: baseline debug/runtime shapes;
- `PhoenixInspect.OptimizedContextTestTarget`: Release/optimized frame and variable limitations;
- W7 duplicate-type and PDB-conflict fixtures: ambiguity and artifact conflict;
- W8 alias, forwarder, request/batch/coordinator/workflow shape targets: binding/authority cross-products;
- `PhoenixInspect.EncTestTarget`: real Hot Reload/Edit-and-Continue generations;
- `Contoso.OrderService`: user-facing deterministic incident.

A fixture should not be “realistic” at the expense of control. It must reach a readiness point only after the fact under test has been established, then pause in a deterministic state suitable for capture.

12.6 The dump test infrastructure

Shared infrastructure starts fixture processes, waits for readiness, captures full or filtered dumps, reopens them, and cleans up. Edit-and-Continue infrastructure compiles baselines and deltas with the pinned Roslyn compiler, applies them through `MetadataUpdater.ApplyUpdate`, verifies edited behavior in-process, and only then exposes readiness.

The infrastructure itself has tests. A broken collector, readiness protocol, environment gate, or process cleanup path could otherwise make product tests pass against the wrong physical state.

12.7 Meaningful synthetic corpus

The headless reference consumer runs “incidents” rather than isolated assertions. Each incident predeclares relevant axes, executes an API path, and emits a stable report. Counterfactual variants deliberately withhold an artifact, change a policy, select another root, or cross a bound. Golden captures the complete contract, including diagnostics and replay identity.

This provides a middle layer between microscopic unit tests and UI demos:

- enough context to prove a feature is useful;
- deterministic enough for byte/golden comparison;
- headless enough for CI and external-host compatibility;
- explicit enough to expose which scenarios remain findings rather than admitted successes.

12.8 Negative evidence and poison cases

Every important exact arm is paired with poisons. Typical poisons include:

- foreign snapshot/module/root identity;
- duplicate metadata names or runtime instances;
- malformed, nil, wrong-kind, or out-of-range tokens;
- truncated memory or metadata/PDB blobs;
- reached cap with cap-plus-one evidence;
- missing/foreign PDB, checksum mismatch, stale disk assembly;
- incompatible receiver/argument/field observation;
- model required but absent/foreign/version-mismatched;
- transition outcome not stamped by the machine;
- Edit-and-Continue chain from mixed histories;
- filtered dump that omits required edit artifacts.

A poison test should assert the exact typed stop and that later capabilities were not consulted. Merely asserting “not exact” leaves room for the wrong layer to compensate.

12.9 Canonical golden tests

Canonical byte tests freeze domains and schema versions. When adding an optional feature, tests should include:

1. absence arm byte-identical to the old encoding;
2. presence arm with an explicit new tag/field;
3. ordering independence where the public contract promises canonical sorting;
4. collision-resistant distinction for semantically different inputs;
5. path/culture/process independence;
6. equality and deterministic hash-code agreement with canonical bytes.

The E2/E3 edit-state work uses this discipline aggressively to prove that unedited modules preserve all previous replay identities.

12.10 Hosted verification at the baseline

The closure record for b2148af6reports:

Lane	Reported passes	Purpose
Unit	1,327	Contracts, planners, machine, domains, product models.
Fast integration	629	Bounded host/product paths not requiring every expensive corpus lane.
Edit-and-Continue fixture	12	Physical edited-process profiles and controls.
Focused delta acquisition	4	E3 exact and poison arms used for closure.
Skipped	0	For the cited closure invocations.

The same commit record states that markdown/link verification passes. These figures are useful orientation, but maintainers should consult the current hosted run or execute the repository wrappers rather than treating a historical count as permanent.

12.11 What a new feature must prove

A robust feature contribution normally needs all of the following:

1. **Contract unit tests:** constructor invariants, equality, canonical bytes, invalid/default inputs.
2. **Semantic unit tests:** exact operation, boundary values, type rules, unsupported shapes.
3. **Capability-order tests:** what must not be called before admission or after a stop.

4. **Poison tests:** partial, foreign, ambiguous, conflicting, and over-bound inputs.
5. **Real evidence:** fixture/dump proving the relevant runtime or artifact fact.
6. **Host projection:** CLI/inspection report retains the product's axes and evidence.
7. **Compatibility:** existing default/absence canonical bytes and corpus reports remain frozen unless intentionally versioned.
8. **Documentation:** authoritative supported surface, limits, and current status updated at the checkpoint that changes truth.

12.12 Fast debugging by test family

Symptom	First test family to run/read
Digest changed unexpectedly	Canonical replay tests for the affected contract, then golden/corpus output.
Correct value, wrong provenance	Provenance*Tests.cs and field/call lineage integration tests.
Method never executes	Request/preparation tests, graph planner tests, then dump acquisition tests.
Unexpected resolver/model call	Planner traversal/accounting and pure-model contract tests.
Static name fails	Parser/binder authority tests before runtime storage tests.
Works in Debug, fails optimized	Optimized-context integration suite and frame/PDB evidence.
Source file not displayed	Portable-PDB source content, frame-source, checksum, and decompilation fallback tests.
Edited module returns stale data	E1 physical-truth, E2 refusal, and E3 delta-acquisition suites.
UI differs from CLI	Inspection service/report tests; neither host should call product internals differently.

Primary code landmarks

Test project roots are `tests/PhoenixInspect.Tests` and `tests/PhoenixInspect.IntegrationTests`. Fixture orchestration lives under `tests/TestInfrastructure`. The headless corpus lives in `src/PhoenixInspect.Headless.ReferenceConsumer`. CI entry points are documented in `.github/workflows/ci.yml` and the `eng/` wrappers it invokes.

Edit-and-Continue and Hot Reload: the current frontier

13.1 The silent-staleness hazard

Edit-and-Continue and Hot Reload can apply metadata, IL, and Portable-PDB deltas to a running module without rewriting the mapped base image. A dump captured afterward may therefore contain:

- current static storage values;
- runtime behavior from an edited method;
- base-image metadata and IL that still describe generation zero;
- runtime edit structures whose public/contract surface is incomplete or version-specific;
- no original delta blob at all;
- no retained PDB delta unless supplied externally.

A base-image evaluator can then return the worst possible result: a stale value or declaration marked exact, with no signal that the module was edited. The active plan treats prevention of silent staleness as independently shippable, even before generation-aware composition is implemented.

13.2 Non-negotiable sequence

The plan imposes this order:

1. measure physical truth in real edited processes;
2. detect applied edits and refuse consumers that would use stale base facts;
3. acquire and validate delta artifacts under exact lineage;
4. compose effective metadata tables;
5. route execution to effective IL and understand edit-added storage;
6. compose PDB deltas only if evidence makes them available;
7. close corpus, compatibility, and documentation.

No consumer's E2 refusal is lifted until the same checkpoint supplies generation-aware authority for that consumer.

13.3 E1: physical truth gate

E1 uses a dedicated hidden fixture. Roslyn EmitDifference produces real metadata/IL/PDB deltas; MetadataUpdater.ApplyUpdate applies them; the process proves the edited behavior before pausing; full and filtered dumps are captured. Profiles include changed bodies, added static fields/types/methods, stacked generations, and changed lexical scopes.

Key measured findings are architecture-changing.

13.3.1 The base image remains generation zero

The mapped metadata bytes and Module row remain byte-identical to the on-disk baseline after verified edits. Generation, generation ID, base generation ID, and base EncLog/EncMap remain generation-zero facts. Therefore any catalog built solely from the mapped image cannot detect edits.

13.3.2 Even a body edit can extend reference tables

A measured changed-body delta included new AssemblyRef and TypeRef rows along with the updated MethodDef. Generation-aware composition cannot special-case “body-only” edits as IL replacement; reference-table extension is part of the metadata truth.

13.3.3 ClrMD surfaces can remain pre-edit

Added fields and accessors were successfully executed in-process yet absent from the runtime type's field/method enumerations. The edited method's IL surface pointed into the mapped base image and exposed generation-zero bytes. These observations rule out treating ordinary ClrMD enumeration/body APIs as effective-generation authority.

13.3.4 The original delta blobs are not runtime-retained evidence

A complete dump-range search found byte-identical metadata and PDB delta copies only as managed-heap residue of the fixture's own payload arrays, after the applying frame was dead and collections had run. No native runtime-retained copy of those original blobs was present. The runtime integrates delta information into internal structures rather than preserving the input blob.

Consequently E3's implemented path is an explicit caller-owned artifact seam, analogous to supplying a PDB. Dump-resident acquisition cannot be claimed until the runtime's internal edit structures are characterized.

13.3.5 Filtered dumps remove edit evidence

The normal filtered capture omitted the delta copies available in the full dump and could also omit descriptor memory required to open the edit-state path. Missing edit evidence is therefore typed unavailable, never “probably generation zero.”

13.3.6 Applied-generation detection is version-pinned

The captured runtime contract descriptor exposes module flags and a dynamic-metadata pointer, but not a named applied-generation count. Measurements found:

- a decoded enablement bit in the declared flags word;
- a counter one pointer-width past the declared dynamic-metadata field that equals one plus the applied-generation count for the pinned runtime;
- unrelated data in that slot for a non-edit-enabled module, so the count is valid only under the runtime’s own edit-enablement gate;
- correct discrimination among edited, edit-enabled-but-unedited, gate-off Debug, and plain modules;
- correct values for zero, one, and two applied generations.

This is physical evidence, but it is a non-contract layout fact and must be validated against the pinned descriptor/toolchain rather than generalized casually.

13.4 E2: detection and typed non-admission

E2 creates an exact per-module edit-state outcome from the enablement fact and physical generation count. Optional descriptor anchors are parsed as optional evidence; their absence produces unavailable rather than a crash or false zero.

The request can carry caller-owned module edit declarations appended in an absence-preserving encoding. Unedited or absent declarations keep old canonical bytes. Binding routes then refuse a module with applied generations before consuming stale base-image authority.

The refusal is wired at multiple boundaries:

- fully qualified owner construction;
- contextual/imported owner construction;
- every ancestry/member-lookup level, including cross-module base levels;
- lexical-envelope completeness for bare/contextual names;
- the real corpus world composer, which physically acquires states for each module and declares them on every evaluation.

A real edited-process test proves the end-to-end exact spelling stops with the edit-specific disposition. An edit-enabled but unedited comparator proceeds to its own independent classification result, proving the edit state—not mere enablement—decides the refusal.

E2 closes silent staleness for the metadata-binding pipeline. IL/body consumers are separate host/execution surfaces and remain part of E5’s refusal/admission work.

13.5 E3: delta metadata acquisition

E3, complete at b2148af60n the artifact-seam arm, validates each supplied metadata delta and composes a lineage chain.

13.5.1 Generation outcome

The outcome retains:

- exact/invalid/partial result kind and typed issue;
- caller-declared ordinal and blob-declared generation;
- MVID, edit ID, base edit ID;
- byte count and blob SHA-256;
- complete physical-order edit-log token/operation rows;
- complete physical-order edit-map tokens;
- canonical bytes and canonical SHA-256.

A byte-count cap is enforced before parser use. Unreadable blobs, ordinal disagreement, and empty edit IDs are prefix-free stops.

13.5.2 Lineage chain

The chain validates every generation outcome, same baseline MVID, root empty-base rule, predecessor edit-ID pairing, generation cap, and optional agreement with the dump's physically acquired applied-generation count. A mixed history from two different edit branches of the same baseline is rejected at the pair join.

13.5.3 Measured vocabulary

The tests pin real `EmitDifference` rows for changed-body and pure-insert generations, including reference-table extension, parent `TypeDef` operations, added field/method/parameter rows, and their map entries. E3 therefore knows what physical edits occurred; it still does not yet calculate effective logical rows.

13.6 E4: generation-aware authority composition

E4 is the next stride at the baseline. It must apply `EncLog/EncMap` operations across the exact chain to produce effective table views while retaining physical history.

Core requirements include:

- updated rows supersede but do not erase prior generations;
- added rows extend RID domains beyond base table ends;
- every token join resolves against effective generation-aware authority;
- definition identity, ownership chains, ancestry, signatures, constraints, member catalogs, and classification are reissued with explicit generation facts;
- zero-generation worlds remain byte-identical;

- each consumer’s E2 refusal is removed only when its effective authority lands.

This will likely be split by table family because TypeDef/Field/Method/Param, references, signatures, and custom/constraint relationships have different edit semantics.

13.7 E5–E7

E5: effective IL and edit-added storage

Find and validate the effective edited method body and exact storage geometry for edit-added statics, or retain typed non-admission where physical evidence remains insufficient.

E6: Portable-PDB deltas

Compose lexical/source generations only through reachable dump evidence or an explicit artifact seam. If no trustworthy delta PDB is available, the E2 lexical refusal remains the final honest behavior.

E7: corpus and closure

Predeclare edited-body, added-field/type, stacked-generation, and filtered-dump incidents; run counterfactuals; freeze conformance/goldens; close traceability and documentation.

13.8 Why this work validates the existing architecture

The Edit-and-Continue findings stress nearly every foundational decision:

- content/source identity prevents base and delta artifacts from being casually mixed;
- independent result axes allow exact detection plus unavailable delta content plus typed refusal;
- complete-table catalogs provide a place to compose generations without losing history;
- parse/select/plan separation lets consumers refuse before touching stale facts;
- canonical absence-preserving encoding protects unedited replay;
- adapter/product separation distinguishes a version-pinned physical detector from general metadata semantics;
- source/PDB separation allows lexical non-admission without disabling unrelated exact storage observations.

The work is not an architectural detour. It is a demanding test of whether “evidence first” has practical consequences when the runtime’s mapped image stops being the whole truth.

Primary code landmarks

The authoritative narrative is `docs/plans/edit-and-continue-support-plan.md`, supplemented by the E1 physical-truth disposition and the exact tests under `PhoenixInspect.IntegrationTests/Enc*.cs`. Implementation contracts include module edit-state types in the `dump-query/ClrMD` surfaces and `MetadataEditGenerationContracts.cs`. At this baseline, E1–E3 are closed and E4 is next.

Extension recipes

This chapter describes the safest route for common changes. The filenames are starting points; exact implementation may span additional partials and contracts. The recurring rule is to extend the lowest layer that owns the new meaning, then carry the result upward without weakening prior outcomes.

14.1 Add an IL opcode

Change recipe

Goal: extend the admitted prepared-graph machine profile with one opcode while preserving planning, typing, events, budgets, provenance, and replay.

14.1.1 Steps

1. **Define the admitted shape.** State operand encoding, stack inputs/outputs, numeric/reference semantics, exceptions, control-flow effect, and whether the instruction is permitted in both legacy and prepared paths.
2. **Decode and validate during planning.** Update body/plan validation so malformed operands, unsupported types, invalid branch targets, and stack-shape contradictions fail before execution.
3. **Extend structural plans if needed.** If the opcode references a field, method, type, or token, freeze the resolved target in `AdmittedMethodPlan`/`FrozenMethodGraphPlan`. Do not leave a raw token for execution-time resolution.
4. **Add the transfer.** Implement one atomic branch in `ILMachine.Transfers.cs` or the relevant prepared partial. Consume exactly one budget unit only on an admitted transfer.
5. **Use domain/memory capabilities.** Do not inspect `ConcreteValue` directly from the generic machine. Add a minimal operation to `IValueDomain` or `IMemoryModel` if the semantics are domain- or memory-dependent.
6. **Emit canonical events.** Preserve the ordinary event-vector rules or explicitly version the product contract if a new event kind is required.

7. **Propagate provenance.** Exact arithmetic/reference transformations should retain operand lineage; precision loss must be explained.
8. **Update graph-derived dynamic cost.** Any loop/control-flow admission changes the finiteness proof and may require a more sophisticated bound than the current acyclic calculation.

14.1.2 Tests

Add machine unit tests for exact transfer, type/stack errors, zero budget, terminal behavior, event vector, and provenance. Add planner tests proving resolution occurs once and unsupported shapes never reach execution. Add counterfactual runner tests proving transition validation accepts the new legal delta and rejects malformed capability behavior. Finally add a compiler-produced fixture method and a real-dump integration test.

Boundary or caveat

A branch or loop opcode is not “just another transfer.” The current runner derives a finite acyclic dynamic instruction bound. Admitting cycles requires a revised proof and result accounting, not merely relying on the caller’s instruction budget.

14.2 Add a pure-call model

Change recipe

Goal: replace one exact versioned helper call with deterministic modeled semantics.

1. Define structural target identity: module/method/signature, not display name alone.
2. Add a model implementation whose inputs/outputs are domain values and whose capability/failure result is explicit.
3. Register it in a versioned catalog with canonical catalog identity.
4. Decide whether callers may opt in generally or require this exact target as part of the request.
5. Ensure planner selection happens before target body acquisition and is frozen in the graph.
6. Record model attempt, entered logical depth, completion, and transformation provenance.
7. Keep interpreted-only requests from consulting the registry.

Tests should cover exact success, unsupported input type, internal model stop, wrong catalog version, required model absent, foreign target, and the capability-order rule. The final result must distinguish modeled effects from ordinary interpreted virtual effects.

14.3 Add a constant/BCL operation

Change recipe

Goal: extend the deterministic folded expression surface without invoking target code or ambient analysis-machine state.

1. Locate the semantic family partial: numeric, sequence, temporal, enum, type, syntax, query, or BCL value.
2. Define exact C# overload selection, conversions, exception/failure behavior, and result type.
3. Identify ambient dependencies. Culture, local timezone, current time, randomness, environment, file system, and mutable global state are inadmissible unless replaced by an explicit deterministic input contract.
4. Use structural target type/enum authority. Do not use same-named analysis-runtime reflection as a shortcut.
5. Bound input traversal and output materialization; use cap-plus-one where exhaustive absence/length matters.
6. Add the operation to completion only when the evaluator can answer it under the active profile.
7. Extend canonical value encoding only if a genuinely new value kind is introduced; otherwise preserve existing encoding.

Differential tests should compare ordinary values with C# or BCL behavior under invariant conditions, plus edge cases such as overflow, NaN, signed zero, decimal scale, invalid enum text, empty sequences, and bound crossings. Add explicit rejection tests for nondeterministic variants.

14.4 Add a new expression syntax form

Change recipe

Goal: admit a Roslyn syntax shape under a new or extended project-owned profile.

1. Parse through the existing pinned front end; do not instantiate a second Roslyn parser.
2. Extend profile classification and immutable descriptor conversion. Decide whether the change is backward-compatible within the current profile or requires a new profile identity/version.
3. Ensure node/token/depth/length bounds are charged consistently.
4. Specify type inference and evaluation order independent of Roslyn's compiler internals.
5. Route operands to existing frozen pipelines where possible; consume typed exact values rather than display forms.
6. Preserve bare-expression fall-through. A new constant pre-stage rule must not change the digest/report of an expression previously answered by a dedicated pipeline unless that change is intentionally versioned.
7. Update completion and documentation only after executable support exists.

Tests should distinguish invalid C#, valid-but-unsupported profile, exact admitted semantics, and expressions that must continue routing to another subsystem.

14.5 Add a new dump value kind

Change recipe

Goal: read and compose a new captured scalar/reference/value-type shape.

The change spans the physical and semantic halves:

1. Add metadata signature/declaration classification under the bounded grammar.
2. Add runtime mapping/decoder expectation to the ClrMD static or instance-field contract.
3. Define exact byte geometry under target pointer width and endianness assumptions.
4. Add counted memory decoding that refuses short reads.
5. Define the product observation/value shape and assignability/layout proofs.
6. Decide whether it can become a root, constant operand, expandable compound, or method-domain value.
7. Add canonical encoding and display projection without using path/culture-dependent text.

Use a dedicated fixture field with boundary values and a sparse-dump poison. For structs, prove field layout from runtime/metadata evidence; do not copy analysis-runtime `Marshal.SizeOf` assumptions.

14.6 Add a ClrMD evidence operation

Change recipe

Goal: expose one new physical observation while containing ClrMD lifetime and uncertainty.

1. Define a PhoenixInspect immutable result/identity type; no ClrMD object may appear in it.
2. Add a session operation with explicit snapshot-scoped inputs and hard caps.
3. Perform all raw reads through the counted memory reader.
4. Map ClrMD/API exceptions and unavailable surfaces to stable adapter statuses/issues.
5. Retain exact prefixes, counters, and source identities.
6. Expose the operation through `DumpInspectionService` or a product facade only if that layer owns its semantics.
7. Invoke it through `DumpSessionHost.QueryAsync`; never from a UI thread directly.

Real-dump tests should include same-toolchain exact evidence, sparse/filtered capture, foreign snapshot input, cap crossing, and lifecycle/thread behavior.

14.7 Add a metadata table family or binding rule

Change recipe

Goal: extend source-anchored metadata authority without introducing first-match or partial-absence errors.

1. Define the physical row contract and complete-table catalog, including source identity, RID coverage, row count, canonical order, and raw fields needed for later proof.
2. Decode with bounded SRM projection and typed malformed/unsupported stops.
3. Define joins by structural tokens and module lineage, not names.
4. Preserve all candidates through ambiguity classification; never discard conflicts before the binder can report them.
5. Add effective-generation facts if the table participates in Edit-and-Continue composition.
6. Extend binder outcomes and completeness certificates carefully. A new lookup path can weaken an old absence proof if its catalog is optional/partial.
7. Add canonical/golden tests for zero-row, exact rows, duplicates, foreign tokens, over-bound table, and generation-zero compatibility.

14.8 Lift an Edit-and-Continue refusal for one consumer

Change recipe

Goal: make one metadata or execution consumer generation-aware while leaving all others conservatively refused.

1. Identify every physical table/body/PDB/storage fact that consumer reads.
2. Build exact effective-generation authority for those facts from the E3 lineage and E4+ composition.
3. Retain superseded physical rows and generation provenance.
4. Prove added and updated rows through real edited fixtures and poison histories.
5. Add an explicit effective-generation fact to the consumer's canonical input.
6. Remove the E2 guard only at that consumer boundary; do not globally mark the module "supported."
7. Prove unedited canonical bytes remain identical.
8. Keep filtered/missing artifact paths typed unavailable.

This per-consumer strategy is slower than a single global feature flag and much safer. Metadata binding, IL execution, static storage, and lexical/PDB context can advance independently according to evidence.

14.9 Add a UI pane

Change recipe

Goal: present existing evidence in a new CLI/desktop view without creating a parallel analysis path.

1. Add or reuse an immutable inspection projection in `PhoenixInspect.Inspection`.
2. Run the adapter/product call inside `DumpSessionHost.QueryAsync`.
3. Ensure projection text reflects exact status/issue/bounds and does not infer missing facts.
4. Add a desktop view model/view or CLI renderer command that depends only on the projection.
5. Clear/recompute pane state when the session, root, frame, or PDB context identity changes.
6. Reuse shared formatting and evaluation-report rendering.

A new pane must not reference `Host.Dump.ClrMD` directly from XAML code-behind or create a second `ClrmdDumpSession`. If genuinely new analysis is needed, implement it below the inspection layer first.

14.10 Add a result or canonical field

Change recipe

Goal: evolve a replayable contract without silently breaking absence/default cases.

1. Decide whether the field changes semantics or merely adds evidence. If semantic meaning changes for existing bytes, bump the canonical schema/profile version.
2. Prefer an appended tagged optional representation whose absence emits no new bytes when byte-frozen compatibility is required.
3. Normalize ordering, casing, and digest representation before writing.
4. Add constructor invariants tying the field to existing axes.
5. Add equality/hash/replay round-trip tests.
6. Freeze old absence bytes as a literal golden and add a distinct presence golden.
7. Update headless corpus and host projection; avoid making display-only expansion part of value identity.

14.11 Update documentation safely

The repository treats milestone evidence as historical. When implementation changes current truth:

- update the active plan/status at the checkpoint that closes the new behavior;
- update the authoritative user-facing supported-surface guide;
- add links from the docs index;

- do not rewrite old closure dispositions as though they had always known later findings;
- record exact tests, counts, caps, and remaining non-admissions;
- keep terminology aligned with diagnostic/result contract names.

Key idea

A PhoenixInspect feature is complete when its failure boundaries are as deliberate as its exact arm. The extension recipe is therefore not “implement, then add tests”; it is “define the evidence contract, admit it under bounds, implement the exact arm and poisons, and expose the unchanged result through hosts.”

A maintainer's reading and debugging guide

15.1 First-day code-reading itinerary

A productive first pass is:

1. `README.md`: understand the user-visible preview and examples.
2. `docs/proposals/architecture/architecture-overview-proposal.md`: learn the truth modes and active data flow.
3. `Core.Abstractions/EvaluationResult.cs` and `EvaluationEvidenceContext.cs`: learn the outcome language.
4. `CanonicalReplayEncoding.cs`: understand why so many contracts retain exact ordering and identity fields.
5. `Host.Abstractions/IProcessMemoryReader.cs`: see the physical-read contract.
6. `Host.Dump.ClrMD/ClrmdDumpSession.cs`: understand world/lifetime ownership.
7. `Product.DumpQuery/StaticFieldExpressionEvaluator.cs`: trace the largest read-only pipeline.
8. `Core.Execution/MethodGraphPlanner.cs` and `ILMachine.cs`: learn the prepare/execute split.
9. `Product.DumpDebugging/CounterfactualMethodRunner.cs`: see product certification around the machine.
10. `Inspection/Services/ExpressionEvaluationService.cs` and `Cli/InspectionSession.cs`: see how hosts remain thin.
11. The matching tests for each file before reading every helper implementation.

This route reveals architecture before scale. Starting with the constant evaluator's many partials or the desktop XAML can make the repository look like an undifferentiated debugger application.

15.2 Search vocabulary

Useful repository-wide search terms and what they reveal:

Search	What to inspect
CanonicalDomain / SchemaVersion	Replay surfaces and compatibility boundaries.
Maximum / BoundReached	Hard caps and cap-plus-one accounting.
FromExact / CreateValidated	Construction gates that encode cross-field invariants.
ToObservationResult	Adapter-to-core evidence projection.
Snapshot != / SnapshotMismatch	World isolation checks.
RejectingExecutionResolver	Proof that execution does not resolve dynamically.
IsIssuedBy	Private authority/stamp checks.
NotConstant	Constant pre-stage fall-through preservation.
AppliedBounds	Which caps are returned and how hosts expose them.
Provenance / Lineage	Source/transform explanations and dynamic propagation.
EditedGenerationsNotComposed	E2 conservative refusal seams.
Exact together with constructor checks	Places where “exact” is proved rather than inferred.

15.3 Debugging by stage

15.3.1 Wrong or missing static-field result

Trace in this order:

1. syntax descriptor and candidate shapes;
2. fully qualified versus contextual binder route;
3. metadata authority catalogs and completeness;
4. selected declaration identity and module edit-state guard;
5. runtime module-instance match count;
6. runtime field mapping and expected decoder;
7. static storage-base/location evidence;
8. raw read bytes/status;
9. nullable layout or reference assignability composition;
10. inspection projection.

Do not jump directly from the expression string to a ClrMD field enumeration. That bypasses the very distinctions the W7/W8 pipeline is designed to enforce.

15.3.2 Method preparation failure

Inspect the preparation result before the run loop:

- canonical request validation;
- binding compatibility;
- planner traversal accounting and first rejected charge;
- body/signature/operand resolution result;
- required model selection;
- graph required logical depth versus configured limit;

- root receiver/argument signature;
- declared field-observation vector order;
- provenance-domain materialization.

A method that never produced a plan is not an execution bug.

15.3.3 Unexpected execution invalidity

Use the diagnostic prefix:

- `W4.Replay.*`: internal/capability result contradicted a frozen invariant;
- `W4.Execution.*`: a capability failed during an otherwise admitted transfer;
- `W4.Budget.*`: deterministic policy prevented completion;
- `W4.Unknown.*`: an admitted unknown/capability path could not continue;
- `W4.Request.*`: public plan/request authority or shape invalid.

Then compare prior/next semantic state, operational budget, event vector, model-attempt delta, and frame-depth delta. The runner's validation code is a checklist of what must remain coherent.

15.3.4 Digest/golden drift

Locate the canonical domain and schema. Compare field write order and normalization. Check whether an optional vector changed from default to empty or whether configured-but-unreached bounds started being serialized. Confirm display expansion did not enter value identity. If the change is intentional, version the contract and update both absence and presence goldens.

15.3.5 Source view mismatch

Separate four questions:

1. Did the frame resolve to an exact module/method?
2. Did a PDB candidate reproduce CodeView identity?
3. Did the PDB map the IL location to a document/span?
4. Did any content source reproduce the document checksum, or is the view explicitly decompiled?

A correct line mapping does not make a mismatching local file authoritative.

15.3.6 Edited-process problem

First inspect the physically acquired module edit state. If generations are nonzero, identify the specific consumer:

- metadata owner/member binding should currently hit E2 refusal unless E4 has lifted it;
- base-image method body use must remain refused until E5 effective IL;
- lexical completeness must remain refused without E6 PDB delta authority;

- externally supplied delta artifacts must match E3 lineage and physical count.

Do not “fix” an edited failure by omitting the edit declaration; that reintroduces silent staleness.

15.4 Performance reasoning

PhoenixInspect prefers explicit finite work over unbounded caching. When profiling:

- distinguish acquisition cost from deterministic product composition;
- inspect caps and actual reached counts before optimizing a traversal;
- preserve first-result caches within one request but avoid cross-world ambient caches;
- do not parallelize ClrMD session work around the dedicated-thread contract;
- ensure a faster path retains the same physical evidence and canonical ordering;
- beware repeated hashing of large artifacts, but never replace content identity with timestamp/path heuristics;
- preserve lazy frame/PDB acquisition for context-independent expressions;
- keep expansion/materialization bounded and separate from parent value identity.

15.5 Code-review checklist

For a substantial change, ask:

1. Which truth mode does the new answer claim?
2. Which physical source and stable identity support it?
3. What are exact, partial, unavailable, conflict, and invalid arms?
4. What finite bounds apply, and how is cap-plus-one represented?
5. Is parsing/resolution/selection done once and frozen before evaluation?
6. Can a path, culture, clock, random source, or analysis-runtime reflection leak into semantics?
7. Are adapter lifetimes and snapshot identities contained?
8. Does the result preserve exact prefixes and provenance?
9. Are default/absence canonical bytes unchanged?
10. Do tests prove capabilities are not called after an earlier stop?
11. Is a real dump/compiler artifact needed to establish physical truth?
12. Can CLI, desktop, and headless hosts use the same inspection/product API?
13. Does current documentation say no more than executable evidence supports?

15.6 Architectural pressure points

Several areas deserve extra caution because they sit at multiple boundaries:

Static-field authority

Joins metadata declarations, runtime module instances, storage geometry, raw bytes, nullable layout, type assignability, suffix evaluation, and contextual PDB facts.

Counterfactual runner

Joins public request identity, private capabilities, graph admission, machine transitions, evidence recording, and canonical result projection.

Source navigation

Joins target debug identity, local artifacts, checksums, optional network capability, and reconstructed presentation.

Edit-and-Continue

Forces base, runtime, artifact, generation, body, and lexical truths to diverge unless explicitly composed.

Constant evaluator

Has broad syntax/BCL coverage and must continually resist ambient host behavior.

Changes here should be staged with small contracts and focused tests rather than a sweeping refactor.

15.7 A final mental model

When following any code path, ask four questions in order:

1. **What world and authority are we in?**
2. **What exact immutable thing was admitted?**
3. **What bounded transformation was performed?**
4. **What kind of claim does the result make, and what evidence did it retain?**

If those answers are explicit, the code is probably aligned with the repository. If one is implicit—especially world identity, admission, or evidence completeness—that is usually where the bug or architectural debt lives.

Production project matrix

Project	Direct PhoenixInspect references	Principal public role	High-value starting files
Core.Abstractions	none	Shared immutable contract vocabulary: result axes, identities, signatures, domains, memory, resolution, replay.	EvaluationResult.cs; EvaluationEvidenceContext.cs; CanonicalReplayEncoding.cs; IValueDomain.cs; IMemoryModel.cs.
Core.Execution	Core.Abstractions	Graph preparation and deterministic IL machine.	MethodGraphPlanner.cs; FrozenMethodGraphPlan.cs; ILMachine.cs; ILMachine.PreparedGraph.cs; ILMachine.Transfers.cs.
Domain.Concrete	Core.Abstractions	Concrete lifted-flat domain, persistent memory, provenance values/lineage.	ConcreteValue.cs; ConcreteDomain.cs; ConcreteMemory.cs; ConcreteMemoryModel.cs; provenance files.
Host.Abstractions	none	Counted immutable process-memory read protocol.	IProcessMemoryReader.cs; MemoryReadResult.cs; MemoryReadStatus.cs.
Metadata.Abstractions	Core.Abstractions	Bounded structural metadata module capability.	IMetadataModule.cs; ModuleId.cs; ModuleDescriptor.cs.
Metadata.SRM	Metadata.Abstractions	SRM/PE acquisition and projection backend.	SrmMetadataModule.cs; SrmMetadataProjection.cs; MetadataResolutionServices.cs.
Host.Dump.ClrMD	Core.Abstractions; Host.Abstractions; Metadata.SRM	Dump/live world, raw memory, modules, heap, frames, PDB, static storage, method resolver.	ClrmdDumpSession.cs and partials; ClrmdProcessMemoryReader.cs; ClrmdDumpExecutionResolver.cs; static contracts.
Product.DumpQuery	Core.Abstractions; Host.Dump.ClrMD	Read-only expression parsing/admission, rooted queries, member/static chains, constant semantics, metadata authority.	DumpQueryParser.cs; DumpQueryEngine.cs; StaticFieldExpressionEvaluator.cs; ConstantExpressionEvaluator.*.cs; edit contracts.
Product.DumpDebugging	Core.Abstractions; Core.Execution; Domain.Concrete; Host.Abstractions; Host.Dump.ClrMD; Product.DumpQuery	Method-path classification, dump import, counterfactual preparation/execution/result.	DumpExpressionEvaluator.cs; CounterfactualMethodRunner.cs; request/plan/result/memory files.
Headless. ReferenceConsumer	Core.Abstractions; Host.Dump.ClrMD; Product.DumpQuery; Product.DumpDebugging	Public API witness, portfolios, corpus, goldens.	Program.cs; UsefulnessPortfolioRunner.cs; W6/W7 portfolio runners.

Project	Direct PhoenixInspect references	Principal public role	High-value starting files
Inspection	Core.Abstractions; Host.Abstractions; Host.Dump.ClrMD; Product.DumpQuery; Product.DumpDebugging	Serialized session, immutable display/report services, completion, source navigation.	DumpSessionHost.cs; DumpInspectionService.cs; ExpressionEvaluationService.cs; SourceNavigationService.cs.
CLI	Inspection	Console capture/open/attach, scripted/interactive inspection transcript.	Program.cs; InspectionSession.cs; ConsoleRenderer.cs; CommandLineOptions.cs.
Desktop	Inspection	Avalonia docked debugger-style shell.	MainWindowViewModel.cs; InspectionDockFactory.cs; Watch/Immediate/source pane view models and views.

Key file atlas

This appendix maps questions to the first source files likely to answer them.

Question	Start here
What combinations of result axes are legal?	<code>src/PhoenixInspect.Core.Abstractions/EvaluationResult.cs</code>
How are snapshot/module identities and reached bounds represented?	<code>EvaluationEvidenceContext.cs</code> ; identity files in <code>Core.Abstractions</code> .
How is replay identity serialized?	<code>CanonicalReplayEncoding.cs</code> ; each contract's <code>CanonicalDomain/schema</code> fields.
What CLI type/signature subset exists?	<code>TypeSig.cs</code> ; <code>BoundedEcmaSignatureProjection.cs</code> .
How is a call graph frozen?	<code>Core.Execution/MethodGraphPlanner.cs</code> ; <code>plan builder/model</code> files.
What happens on one IL instruction?	<code>ILMachine.Transfers.cs</code> ;
What is concrete unknown/bottom/exact?	<code>ILMachine.PreparedGraph.cs</code> ; <code>event/state</code> files. <code>Domain.Concrete/ConcreteDomain.cs</code> ;
How are persistent field reads represented?	<code>ConcreteValue.cs</code> .
How is a PE module opened and identified?	<code>IMemoryModel.cs</code> ; <code>ConcreteMemoryModel.cs</code> ; recording memory in <code>Product.DumpDebugging</code> .
How are method/field signatures projected?	<code>Metadata.SRM/SrmMetadataModule.cs</code> .
How is a dump or live process opened?	<code>Metadata.SRM/SrmMetadataProjection.cs</code> .
How are target bytes read?	<code>Host.Dump.ClrMD/ClrmdDumpSession.cs</code> .
How do runtime methods become execution definitions?	<code>Host.Abstractions/IProcessMemoryReader.cs</code> ;
How is a frame/PDB context acquired?	<code>ClrmdProcessMemoryReader.cs</code> . <code>ClrmdDumpExecutionResolver.cs</code> ;
	<code>ClrmdMethodBodyParser.cs</code> . <code>ClrMD expression-context/source</code> partials and <code>Portable-PDB</code> resolver contracts.

Question	Start here
How is a fully qualified static field evaluated?	Product.DumpQuery/StaticFieldExpressionEvaluator.cs.
Where is metadata name binding implemented?	StaticFieldFullyQualifiedBinder.cs; contextual binder and metadata-authority files.
How does the original rooted field query work?	DumpQueryParser.cs; DumpQueryEngine.cs; DumpQueryPlan.cs.
Where are member chains handled?	member-chain parser/plan/engine files in Product.DumpQuery and facade files in Product.DumpDebugging.
Where is folded C# semantics implemented?	ConstantExpressionEvaluator.cs and its partials by semantic family.
How are query and method paths classified?	Product.DumpDebugging/DumpExpressionEvaluator.cs; classifier/request files.
How is a method plan issued and executed?	CounterfactualMethodRunner.cs; request/preparation/plan/result files.
How are UI reports made without changing semantics?	Inspection/Services/ExpressionEvaluationService.cs; Models/EvaluationReport.cs.
How is ClrMD serialized onto one thread?	Inspection/Infrastructure/DumpSessionHost.cs.
How is source verified or decompiled?	Inspection/Services/SourceNavigationService.cs; PortablePdbSourceContent.cs.
What commands does the CLI support?	Cli/InspectionSession.cs; Cli/Program.cs.
How are Watch and Immediate built?	Desktop Watch/Immediate view models, views, and shared evaluation service.
How is Edit-and-Continue state detected?	ClrMD/module edit-state contracts, E1 tests, and docs/plans/edit-and-continue-support-plan.md.
How are delta blobs and lineage validated?	Product.DumpQuery/MetadataEditGenerationContracts.cs; IntegrationTests/EncDeltaAcquisitionTests.cs.



Command and workflow reference

C.1 Build and preview

```
# Deterministic preview incident: build target, capture dump, replay transcript
./eng/Invoke-PreviewDemo.ps1

# Build the solution with the pinned SDK/package locks
dotnet restore PhoenixInspect.sln --locked-mode
dotnet build PhoenixInspect.sln -c Release --no-restore

# Open an existing dump through the CLI
dotnet run --project src/PhoenixInspect.Cli -- <path-to-dump>

# Attach to a suspended live process
dotnet run --project src/PhoenixInspect.Cli -- --attach <pid>
```

Exact wrapper/test commands can evolve; the CI workflow and eng/ scripts are the authoritative executable source at a given commit.

C.2 CLI startup options

Option	Meaning
--attach <pid>	Attach to a running compatible .NET process; target remains suspended until session close.
--eval <expression>	Evaluate one expression in scripted order; may repeat.
--command <text>	Execute one session command; may repeat.
--script <path>	Read commands one per line; comments begin with ##.
--verbose	Print complete evidence behind evaluations.
--no-color	Disable ANSI styling.
--help	Print usage.

C.3 Interactive commands

Command	Purpose
info / overview status	Snapshot identity, target, bounds, expression profile. Current root, root identifier, frame context, PDB candidates, verbosity.
modules [substring]	List/filter runtime managed module instances.
module <index>	Read counted metadata identity for one module.
threads [probe] [depth]	Probe thread ordinals and show managed frames.
frames <thread> [count]	Load bounded managed stack for one thread ordinal.
context <thread> <frame>	Adopt exact frame for contextual name binding.
context <method text>	Find first probed frame whose method display contains text.
context none	Withdraw contextual namespace/import/alias evidence.
pdb <path> / pdb auto / pdb clear	Manage bounded Portable-PDB candidate set.
objects <TypeName> [max]	Bounded strong-handle search by exact ordinal runtime type name.
root <index>	Adopt a prior search match.
root <static expression>	Evaluate static field and adopt its exact object reference.
root none	Withdraw root.
as <identifier>	Rename root identifier.
verbose on off	Toggle full evidence.
eval <expression> / bare expression	Evaluate through routed Watch semantics.
exit / quit / q	Close session and resume live target if attached.

C.4 Capture

```
phoenixinspect capture --pid <processId> --output <path>
```

The CLI capture command writes a full dump. PhoenixInspect does not require dumps to originate from its own collector; identity and adapter admission apply to any supplied dump.

Outcome and evidence reference

D.1 Core evaluation axes

Axis	Value	Interpretation
Semantic mode	Observation	Direct captured/runtime/artifact fact.
	DerivedQuery	Deterministic composition/binding over observations.
	CounterfactualExecution	Interpreted/modelled computation over virtual imported state.
	AbstractAnalysis	Abstract semantic result contract.
Completion	Completed	Operation reached its normal terminal contract.
	Blocked	Required admitted capability/evidence was unavailable or unsupported.
	BudgetExhausted	A deterministic configured bound prevented further progress.
	Cancelled	Host cancellation observed at an allowed boundary.
	Invalid	Input, artifact, program, or internal capability result contradicted the contract.
Completeness	Complete	Full semantic value is present.
	Partial	A meaningful prefix/partial value is present.
	None	No semantic value is claimed.
Evidence	Exact	Every required physical input was complete and coherent.

Axis	Value	Interpretation
	Partial	A known prefix/subset was available; exhaustion is not claimed.
	Unavailable	Required evidence could not be acquired.
	Conflict	Independently valid facts disagreed or identity joins failed.
	Invalid	Evidence bytes/shape were malformed or self-contradictory.
Effects	None	Read-only observation/derived semantics.
	VirtualOnly	Effects occurred only in descendant persistent virtual state.
	Modeled	A declared pure model contributed semantic behavior.
	Unsupported	Effect shape is outside the admitted contract.

D.2 Common host labels

The UI may render compact labels such as exact, absent, partial, stopped, or rejected. These labels are projections, not the canonical core enum. Read the underlying axes and diagnostic for the precise distinction.

D.3 Adapter evidence

ClrMD/product-specific enums add domain detail: exact/unavailable/partial/conflict/invalid statuses, value issues such as module unavailable, ambiguous match, bound reached, unsupported type/layout, or sparse read, and stage enums identifying syntax, symbol binding, runtime declaration, nullable layout, storage, assignability, suffix, or complete composition.

A stage indicates the last boundary reached. It must not be read as a severity ordering: an exact syntax result followed by context-required binding is useful exact evidence about why no value exists.

D.4 Deterministic bounds

A bound has a stable name and numeric value. Only a bound actually applied/reached belongs in the result context. Common categories include expression characters, syntax nodes/tokens/depth, identifier/string lengths, graph traversal units, method count, instruction units, logical depth, field/handle/module scans, raw read bytes, artifact bytes, source size, result/sequence materialization, edit generations, and delta blob bytes.

Supported expression surface at the baseline

This is an orientation summary, not a substitute for the repository's authoritative preview quickstart and tests.

Family	Representative admitted behavior	Representative deliberate stop
Static fields	Fully qualified metadata binding; contextual names with exact frame/PDB imports; scalar/reference/nullable values.	Missing/ambiguous module or declaration; edited generation not composed; sparse storage.
Root member chains	Arbitrary admitted depth, conditional access, null coalescing, strings/scalars/references.	Unavailable hop; unsupported runtime field kind; exact null under ordinary dot.
Numeric	Fixed integers, 128-bit integers, BigInteger, float/double, decimal, promotions, checked semantics.	Unsupported/user-defined conversions; nondeterministic or unmodelled APIs.
Strings/chars	Deterministic ordinal/culture-independent surface, indexing/ranges, invariant formatting.	Culture-sensitive behavior without explicit invariant contract.
Arrays/sequences	Creation, initializers, spreads, dump arrays, pure System.Array and Enumerable operations, lambdas.	Mutators such as in-place sorting; over-bound enumeration/materialization.
Temporal	Date/time/span/date-only/time-only constructors, ticks, arithmetic, comparison, invariant formats.	Now, local timezone, culture-dependent parsing.

Family	Representative admitted behavior	Representative deliberate stop
Guid/Version	Fixed-grammar parse/construction, comparison, invariant format.	Fresh randomness such as Guid.NewGuid.
Enums/types	Casts, flags, Enum API, typeof, Type queries, assignability, generics/variance under metadata authority.	Missing/incomplete target metadata; reflection-only ambient facts.
Queries/anonymous	from/where/let/order/group/join/ continuations and structural anonymous values.	Operator/type shapes outside deterministic folded surface.
Modern syntax	Interpolation, selected patterns/switch expressions, nameof/default/sizeof/checked wrappers.	Unsupported pattern kinds or target-code invocation.
Method expressions	Closed prepared IL profile, direct admitted calls, optional versioned pure models.	Unsupported opcode/signature/call, missing evidence, budget/depth, edited body not composed.

Glossary

Admission	Conversion from syntactically/physically possible input to a closed, versioned shape the product promises to evaluate.
Applied bound	A deterministic cap that materially constrained or was reached by the operation; distinct from a merely configured maximum.
Artifact identity	Content-derived identity of a complete external file, usually SHA-256 based.
Canonical bytes	Domain-separated, versioned, prefix-free deterministic encoding used for equality/replay digests.
Cap-plus-one	Traversal technique that observes one item beyond a nominal retention cap so exact completion and truncation can be distinguished.
Conflict	Two otherwise valid identities/facts disagree; not the same as malformed evidence.
Counterfactual	Computation in an imported virtual state describing what admitted IL/model semantics produce, not a historical observation.
Derived query	Read-only deterministic composition over observations, such as member binding or arithmetic over exact dump values.
Descriptor	Non-identity display/location facts associated with a stable identity.
Evidence prefix	Exact facts established before a later partial/unavailable/conflicting stop.
Exact absence	Exhaustive proof that no matching item exists within a complete admitted universe; distinct from zero retained results after a bounded scan.
Host	CLI, desktop, headless, or other caller that presents product/adaptor outcomes through the Inspection layer.
Lineage	Provenance graph connecting a value/result to dump fields, calls, models, and transformations.
Module instance	One runtime load of module content in a particular runtime/domain context; multiple instances may share content identity.
Observation	A direct fact from dump/runtime/artifact evidence.
Physical truth gate	Real-process/artifact experiment performed before designing an admission contract for uncertain runtime behavior.
Plan	Immutable selected/admitted execution or query input whose identity freezes resolution before evaluation.

Prefix-free stop	Non-exact outcome constructed at the first failed boundary without fabricating later-stage fields.
Provenance	Named source or transformation supporting a result: dump memory, runtime structure, artifact, policy, transformation.
Pure model	Versioned deterministic semantic replacement for one exact call target, selected during planning.
Replay identity	Canonical digest identifying the complete semantic/evidence/policy contract of an operation/result.
Root	Exact snapshot-scoped object binding adopted as the starting point of root-relative expressions.
Snapshot identity	Identity of the captured world: complete dump content hash or live-attach circumstances identity.
Typed non-admission	Deliberate result saying a capability is not admitted because required evidence/composition is unavailable, rather than guessing.
World isolation	Requirement that roots, frames, modules, and plans from one snapshot cannot be reused against another.

Baseline and source methodology

G.1 Exact baseline

This walkthrough describes the repository's main branch at:

`b2148af615d0a0b2694c8515f4beb99151f954f7`

The commit was authored on 6 August 2026 UTC and closes the E3 delta-metadata acquisition exit gate on the artifact-seam arm. Later commits may change APIs, supported surfaces, project counts, status text, and verification counts.

G.2 Primary material inspected

The walkthrough was produced from:

- the complete repository tree and solution/project files;
- root README, documentation index, architecture proposal, active plans, and milestone/status records;
- representative public contracts and high-value implementation files in every production layer;
- unit/integration fixture trees and test names;
- CI workflow and package/build configuration;
- recent commit metadata and closure verification record.

The analysis intentionally prioritizes exact source and active-plan status over stale summary prose. Where a statement is based on repository-reported hosted test results rather than an independently executed run, the text says so.

G.3 Repository links

- Repository: [PhoenixInspect on GitHub](#)

- Exact commit: [b2148af615d0a0b2694c8515f4beb99151f954f7](#)
- Source tree at baseline: [commit-pinned tree](#)

Closing perspective

PhoenixInspect's code volume comes less from the raw act of reading a field or interpreting an instruction than from refusing to conflate neighboring truths. A module name is not a module identity. A mapped image is not necessarily the effective edited module. A PDB path is not a PDB identity. A source path is not source content. A short read is not zero. A counterfactual return is not historical behavior. A bounded miss is not absence. A prepared-looking object is not an authorized plan.

Once those distinctions are internalized, the repository becomes much more regular. Every layer asks for an exact source, admits a finite shape, freezes an immutable identity, performs a bounded transformation, and returns a result that says what it knows and what it does not. That regularity is the main asset to preserve as the evaluator grows.

List of Figures

1.1	The repository's recurring control-flow shape. Evidence is acquired before it is composed; the final result retains both the composition and its physical inputs.	3
2.1	Layered production dependency direction. The dashed shortcut represents the headless consumer's direct use of product APIs; Appendix A lists every exact project reference.	6
3.1	Conceptual shape of canonical contract encoding. The exact writer API is typed, but every sequence is domain-separated, versioned, and prefix-free.	14
4.1	The plan/execute split. No metadata or call-target resolution is permitted in the execution loop.	16
6.1	Measured Edit-and-Continue lineage relation retained by the E3 contracts.	28
7.1	Host-side serialization of every ClrMD operation. Only immutable PhoenixInspect projections return to UI code.	33
9.1	Private issuance prevents a caller from assembling an executable-looking plan from public data.	44
10.1	Source content is shown only through a proved identity/checksum path; decompiled C# is a separately labelled reconstruction.	49